

UNIFIED MODELING LANGUAGE

(UML)

The logo for the Unified Modeling Language (UML) features the acronym "(UML)" in a white, serif font with a subtle drop shadow. Below the letters "U" and "M" are two stylized, orange-colored symbols that resemble calligraphic flourishes or the letters "d" and "c" respectively.

UML

- ☞ Sistem geliştirmede nesneye yönelik yaklaşım, Tümleştirilmiş Modelleme Dilinin (**UML**: Unified Modeling Language) standartlaştırılmasıyla 1990'larda yaygınlaşmaya başlamıştır. Klasik yaklaşımın tersine, veri merkezli, sınıf modelleri üzerinde gerçekleştirilen bir yaklaşımdır.

UML

Nesneye yönelik analiz ve tasarım yöntemleri kullanıldığında;

- ☞ Kodların ve modüllerin yeniden kullanılabilirliği kolaylaşır,
- ☞ Geliştirme süresi kısalır,
- ☞ Üretkenlik artar,
- ☞ Yazılım kalitesi yükselir,
- ☞ Anlaşılabilirlik kolaylaşır.

UML

Nesneye yönelik yaklaşım ile;

- ✎ Yazılım geliştirilirken çeşitli süreç modelleri uygulanabilir.
- ✎ Kabul görülen en iyi özellikler bir araya getirilerek, modelin yapısına uygun birleşik (tümleştirilmiş) yazılım süreci (Unified process:**UP**) ortaya atılmıştır.
- ✎ UP, tekrarlı ve artımlı bir süreç yapısı çizer,
- ✎ Tek bir model, analiz, tasarım, gerçekleştirme ve test aşamalarında sürekli işlenir, değiştirilebilir ve iyileştirilebilir.

UML

- ❧ Geleneksel yazılım geliştirme metotlarının eksikliklerini mümkün olduğunca azaltan nesneye yönelik yaklaşım, birkaç yeni problemi de beraberinde getirmiştir.
- ❧ Sistemin gerektirdiği çözüm ilişkisel bir veritabanı kullanımı gerektiriyorsa, kavram ve uygulama arasında belirgin farklar oluşur.
- ❧ Süreç içinde aşamaları birbirinden kesin olarak ayırmak zordur.
- ❧ Bu da yazılım ölçümlerinin yapılabilmesini zorlaştırır.
- ❧ Ancak projenin küçük parçalara bölünerek yönetilmesi ve yinelemeli-artımlı süreç bu problemi çözmeye yarar.

- ✎ Nesneye yönelik sistem analizi ve tasarımda UML bir standart olarak yaygın kullanılmaktadır.
- ✎ UML,bir yöntem değil,modelleme dilidir.
- ✎ UML ile tasarım anlaşılır bir biçimde ortaya koyulabilir.
- ✎ Notasyon, ağırlıklı olarak çizimlerden oluşur.

- Bir kullanıcı için önemli olan sistemin nasıl geliştiğinden çok, doğru çalışması ve istediği işi yapmasıdır.
- Zamanla değişen istekler karşısında gerekli değişiklikleri ve yenilikleri kolayca yapabilmenin yolu sistemin daha tasarım aşamasında iyi tanımlanmasıdır.
- Ayrıca analiz ve tasarım sırasında çizilen UML diyagramları ile sistemin resmi çizilerek, müşteri ile etkileşim içinde, projenin doğru yönde ilerlemesi sağlanır.

NESNEYE YÖNELİK TEMEL KAVRAMLAR

- ☞ Gerçek dünyada nesneler, belli sorumlulukları yerine getiren varlıklardır.
- ☞ Yazılımda ise değişik hizmetler veren ve özellikleri olan bir varlık olarak düşünülebilir.
- ☞ Programcı gözüyle bakıldığında ise nesne, veri ve fonksiyondan oluşan bir modülden başka bir şey değildir.

- ✎ Bilgi sistemi geliřtirmenin amacı, bir problemi çözmektir.
- ✎ Nesneye yönelik yaklaşımda problemi oluřturan nesneler, gerçekteki yapılarına benzer olarak modellenir ve modellenen nesnelerin de özellikleri ve davranışları olur.
- ✎ Gerçek hayattan bir örnek verirsek;
 - Bahçemizdeki ağaç bir nesnedir.
 - Yaprak türü, gövde uzunluğu, meyveleri, çiçekleri nesnemizin özellikleri,
 - Ağacın büyüyerek çiçek açması yaprak ve meyve vermesi de ağacın davranışlarıdır.

- ✎ Nesneler birbirlerinden kimlik bilgileri ile ayrılırlar.
- ✎ Örneğin; aynı yemek takımına ait iki tabak tıpatıp aynı gibi gözükse de üretim sırasında farklı seri numaraları verildiyse farklı seri numaralarına sahip farklı iki tabak olurlar.
- ✎ Nesneye yönelik yaklaşımın hedefi, gerçek hayatta nesnelerin sahip olduğu durum, davranış, kimlik gibi özellikleri örnekleme ve modellemeye çalışmaktır

UML Diyagramları

Yapısal UML Diyagramları

1. Sınıf Diyagramı (*Class Diagram*)
2. Bileşen Diyagramı (*Component Diagram*)
3. Birleşik Yapı Diyagramı (*Composite Structure Diagram*)
4. Nesne Diyagramı (*Object Diagram*)
5. Paket Diyagramı (*Package Diagram*)
6. Yerleştirme Diyagramı (*Deployment Diagram*)

UML'in Davranış Diyagramları

1. Kullanım Senaryosu Diyagramı (*Use-Case Diagram*)
2. Faaliyet Diyagramı (*Activity Diagram*)
3. Durum Diyagramı (*State Machine Diagram*)
4. Etkileşim Diyagramları
 1. Ardışıklık Diyagramı (*Sequence Diagram*)
 2. Zamanlama Diyagramı (*Timing Diagram*)
 3. Etkileşime Bakış Diyagramı (*Interaction Overview Diagram*)
 4. İletişim Diyagramı (*Communication (Collaboration) Diagram*)₁₁

Modelleme Elemanları

- ☞ **Aktör (Actor)**; Sistemin kullanıcılarını tanımlamak için kullanılır. Bir insan, bir başka sistem veya bir cihaz olabilir.
- ☞ **Kullanım Senaryosu (Use Case)**; sistemdeki aktörler için, belirli bir sonuç değerini oluşturan, bir dizi eylem bütünüdür. İşlevsel isteklerin anlaşılmasına yardımcı olan bir yöntemdir.
- ☞ **Yapısal Elemanlar (Structural Elements)**; sistemde bulunan yapıtaşlarını, yani sınıfları, nesneleri ve paketleri içeren statik kısımlardır.
- ☞ **Davranışsal Elemanlar (Behavioral Elements)**; kullanılan eylem sözcükleri yardımıyla yazılım modelinin dinamik kısımlarını tanımlar.
- ☞ **Gruplama Elemanları (Grouping Elements)**; diğer adıyla paketler, yazılım elemanlarını gruplamak amacıyla kullanılırlar. Paketlerin içinde sınıflar, arayüzler, diyagramlar, hatta başka paketler bulunabilir.

Modelleme Elemanları

- ☞ **Nesne (Object);** yazılım modeli içindeki belirli ve sonlu sayıda olan sistem elemanlarıdır. Nesnelerin karakterleri ve bu karakterlerin üzerinde duran davranışları vardır. Nesneler bu yapılarıyla (karakter, davranış) sistem üzerinde kendilerine verilen görevi yerine getirirler.
- ☞ **Sınıf (Class);** nesnelerin sistem içerisinde oynadıkları rolü, yani nesnelerin öznitelikleri ile iş yapan metotlarını kapsayan yapılarını belirlerler. Sınıflar nesneler için bir kalıp görevi görürler. Sistem içerisinde nesneler, bu kalıba göre oluşturulurlar, belirli etkinlikleri yerine getirirler ve sonlanırlar. Sınıflar, bir yandan varlıkların belirli özellikleri ve davranışlarına odaklanarak soyutlama sağlarken diğer yandan varlıkların bazı özelliklerini ve davranışlarını kapsayıp dış dünyadan gizleyerek karmaşıklığı azaltırlar.

Modelleme Elemanları

- ☞ **Aktif Sınıf (Active Class);** kendi başlarına var olabilen, uygulamanın akışını kontrol edebilen nesneleri tanımlarlar.
- ☞ **Arayüz (Interface);** nesnelerin davranışlarını belirleyen kurallar bütünü olarak düşünülebilir. Sınıflar, davranışları belirleyen arayüz yordamlarını gerçekleştirirler.
- ☞ **İşbirliği (Collaboration);** belirli bir amaca yönelik kullanım senaryolarının birleştirilmiş halidir.
- ☞ **Bileşenler (Component);** yazılım gereksinimlerinin belirli kısımlarını gerçekleştiren modüllerdir.
- ☞ **Düğüm (Node);** sisteme hesaplanabilir ve ölçülebilir durumda olan bir kaynak sunan fiziksel elemanlar.

1.) Sınıf (Class)

∞ **Sınıf**, aynı özellik ve işlemlere sahip nesne kümesi için bir tanımlayıcıdır. Nesnelerin oluşturulması için şablon olarak da düşünülebilir.

Bir sınıfın UML gösterimi;

Sınıf Adı
Özellikler
İşlemler():

A.) Özellikler (Attribute)

∞ **Özellik** bir tip-değer çiftidir. Sınıflarda özelliklerin tip tanımları yapılır, nesnelerde ise bu özellikler değer taşırlar.

Özellik isimleri ve tiplerinin belirtildiği gösterim örnekleri;

Mimar	Yapı
Ad: String	Ad: String
Soyad: String	Yer: String
Doğum Tarihi: Date	Yapım Tarihi: Date
Ölüm Tarihi: Date	Yapan: Mimar

- ☞ Bir sınıfın özellikleri yalnızca basit veri tiplerinden oluşabileceği gibi başka sınıftan tanımlanmış nesnelerde olabilir. Örneğin “Mimar” sınıfı basit veri tiplerine sahipken, “Yapı” sınıfının “Yapan” adlı özelliği “Mimar” sınıfı tipindedir. Sınıflar arasındaki bu ilişki, “**sahip olma**” (has a) ilişkisi olarak adlandırılır.

- ☞ Nesneler, birbirleriyle mesajlar yoluyla haberleşirler.
 - Mesaj, sınıfta tanımlı bir işlemin harekete geçirilmesini sağlar.
 - Bir nesneyi kullanmak için iç yapısının bilinmesi gerekli değildir ancak nesnenin dışarıya sunduğu hizmetlerin nasıl kullanılacağına bilinmesi gerekir.
- ☞ Nesnenin özelliklerine direk erişim yapılmaması veri tutarlılığı açısından önemlidir. Bunu sağlamak için sınıflar yazılırken bazı üyeler gizlenir, dışarıdan erişimleri engellenir.

Bir sınıfın üyelerine erişim için üç etiket kullanılmaktadır. Bunlar;

- ☞ **Public (açık):** Üyeler diğer nesnelerin kullanımına açıktır. UML'de üyenin başına konulan “+” işareti ile gösterilir.
- ☞ **Private (özel):** Üyeler sadece o sınıfın nesneleri tarafından kullanılabilir. UML gösterimi “-” işaretidir.
- ☞ **Protected (korumalı):** Üyeler sadece o sınıfın nesneleri ve alt sınıflarından türeyen nesneler tarafından erişilebilir. UML gösterimi ise “#” işaretidir.

B.) İşlemler (Method)

- ☞ Nesne, bilgi saklayan özelliklerinin yanı sıra bu özellikler üzerinde işlem yapmaya yardımcı olan işlemleri de içerir.
- ☞ Bir işlemi gerçekleştiren yordama ise, “**metot**” denir.
- ☞ Metotlar; kendisiyle aynı isimde olan mesajla aktif hale gelir, bu mesajda metodun sahip olduğu parametrelerin alacağı değerler de yer alır.

İşlemler nesnelerin birlikte çalışmasını sağlarlar

Yığın
- içerik: Liste
+ push(): + pop():

Liste
+ add(): + remove(): + first(): + last():

∞ Bir işi gerçekleştirmek için, birçok nesne aralarında mesajlaşarak işbirliği yaparlar. Yukarıda “Yığın” sınıfı, “push()” işlemini yapabilmek için “Liste” sınıfının “add()” hizmetinden yararlanır.

- ✎ İşlemlerin görülebilirliği de özelliklerin görülebilirliği ile aynı esaslara dayanır.
- ✎ Nesne yönelimli bir programda dış dünyaya hizmet eden ve haberleşen metotlar açık görünürlüğe sahip, sınıfın kendi iç yapısında kullandığı işlemler ise özel görünürlüğe sahip olup dışarıdan erişilemezler.

Statik = Sınıf Diyagramları (Class Diagram)



Erişilebilirlik (Visibility)	Değişken İsmi	:	Tür İsmi	[= ilk değer]
---------------------------------	---------------	---	----------	-----------------

```
Public id : Integer = 0
Private mKod : HashTable
```

Erişilebilirlik	Metot İsmi	([parametre listesi])	:	[Geri Dönüş Değerinin Türü]
-----------------	------------	---------------------------	---	-------------------------------

```
Private MoveLastRecord() : DataRow
```

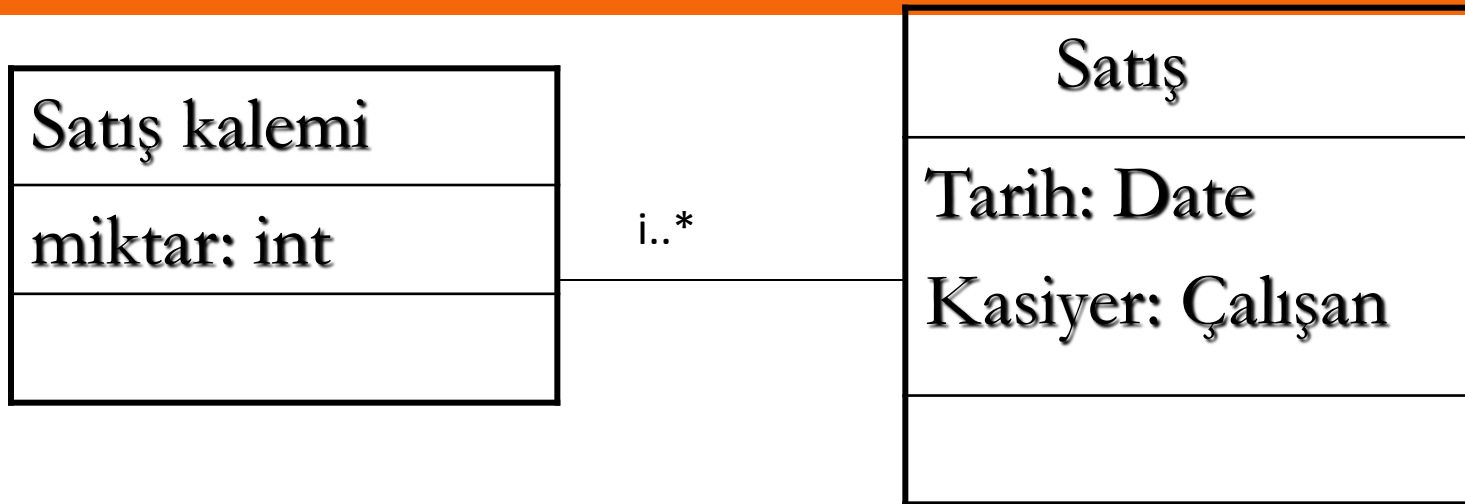
2.) Sınıflar Arası İlişkiler (Relations)

- ☞ Nesneye yönelik bir uygulamanın analiz aşaması, temel olarak sınıfların ve sınıflar arasındaki ilişkilerin belirlenmesi esasına dayanır.

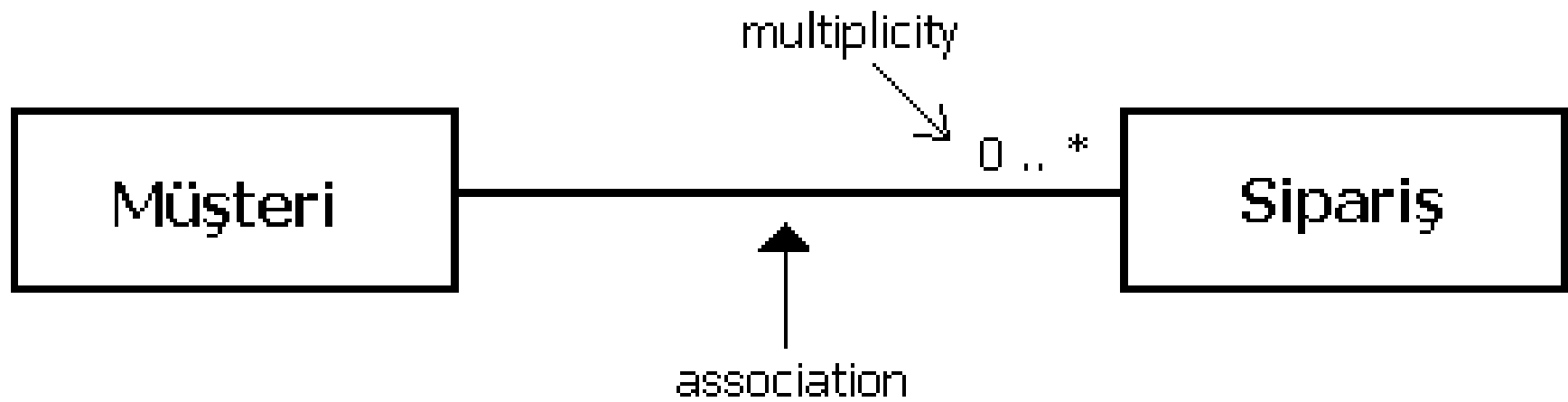
A.)Bağlantı(Association)

- ∞ **Bağlantı**, sınıflar arası ilişkilerin bir çeşididir.
- ∞ Sınıflardan türeyen nesneler arasında hizmet alışverişini sağlar.
- ∞ Birbirleriyle haberleşecek nesneler bu bağlantıyı kullanırlar.
- ∞ Bağlantılarda genellikle iki sınıf yer alır; ancak tek sınıf da kendi üzerinde bağlantıya sahip olabilir.

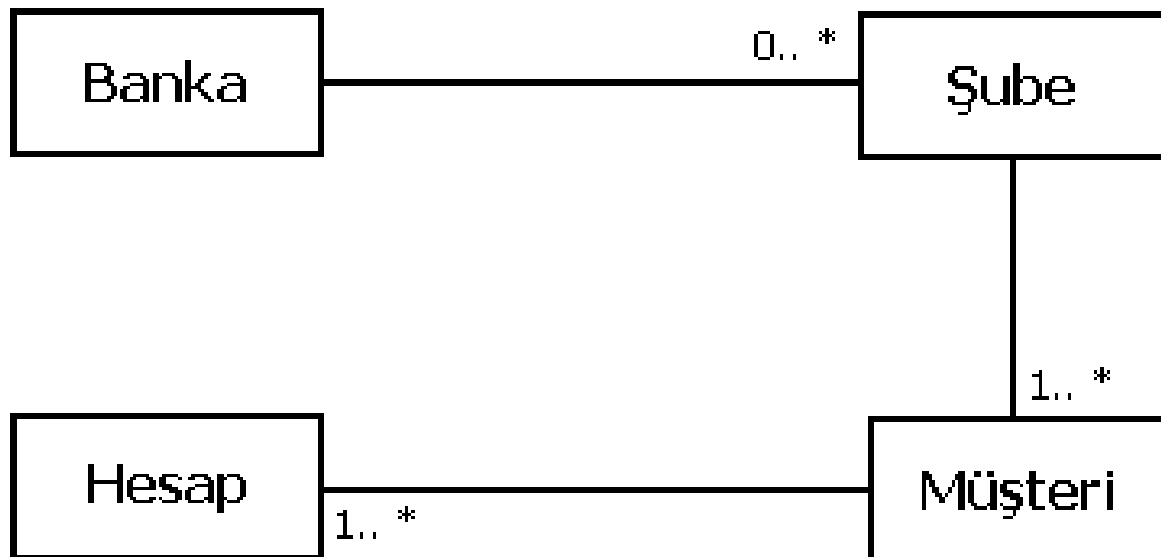
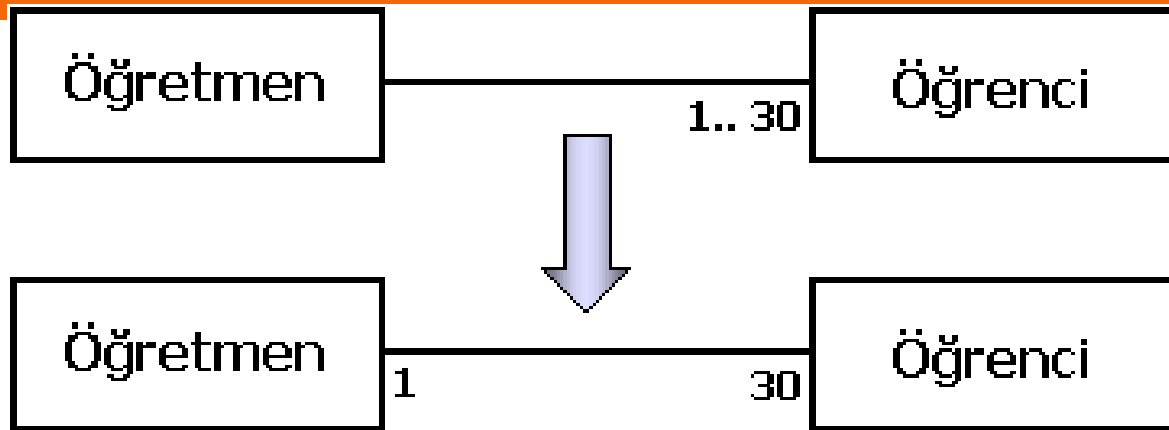
Sınıflar arası bağlantı



- ∞ n1...n2 arası değişen tamsayılarla gösterilen çoğullama sayısı, kaynak sınıfın bir nesnesiyle hedef sınıfın kaç nesnesinin ilişkilendirilebileceğini gösterir.
- ∞ Sayısı kestirilemeyecek kadar çok çoğullama varsa, bu sayı yerine “*” işareti kullanılır.

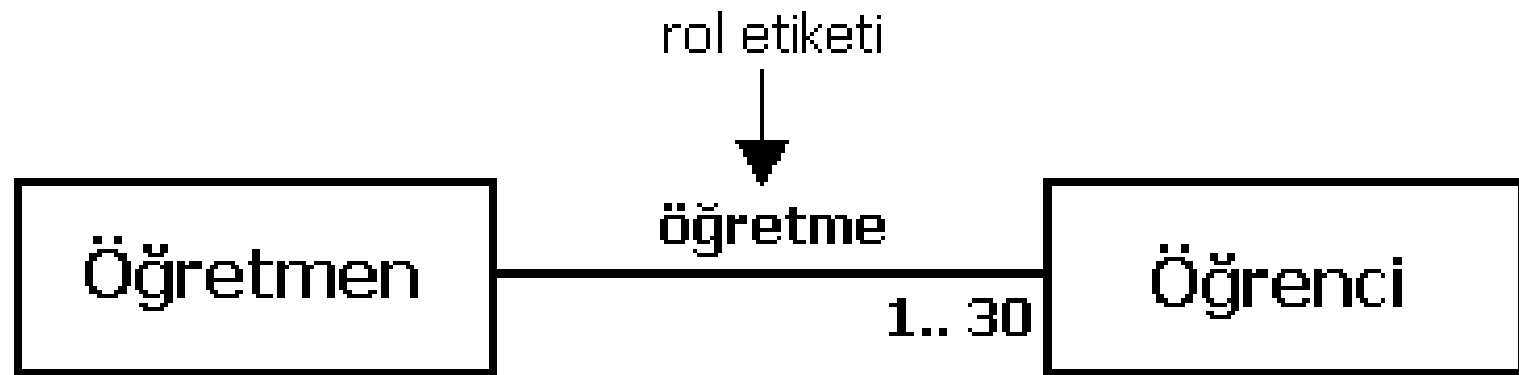


Multiplicity/Cardinality



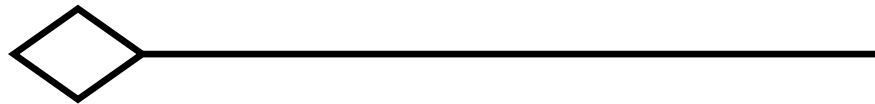
1	1 adet
*	Belirsiz çokluk
0.. 1	Var ya da yok
1.. *	1 tane ya da sonsuz sayıda
m.. n	2 değer arasında
0.. *	Hiç yok ya da sonsuz sayıda

Rol Etiketleri



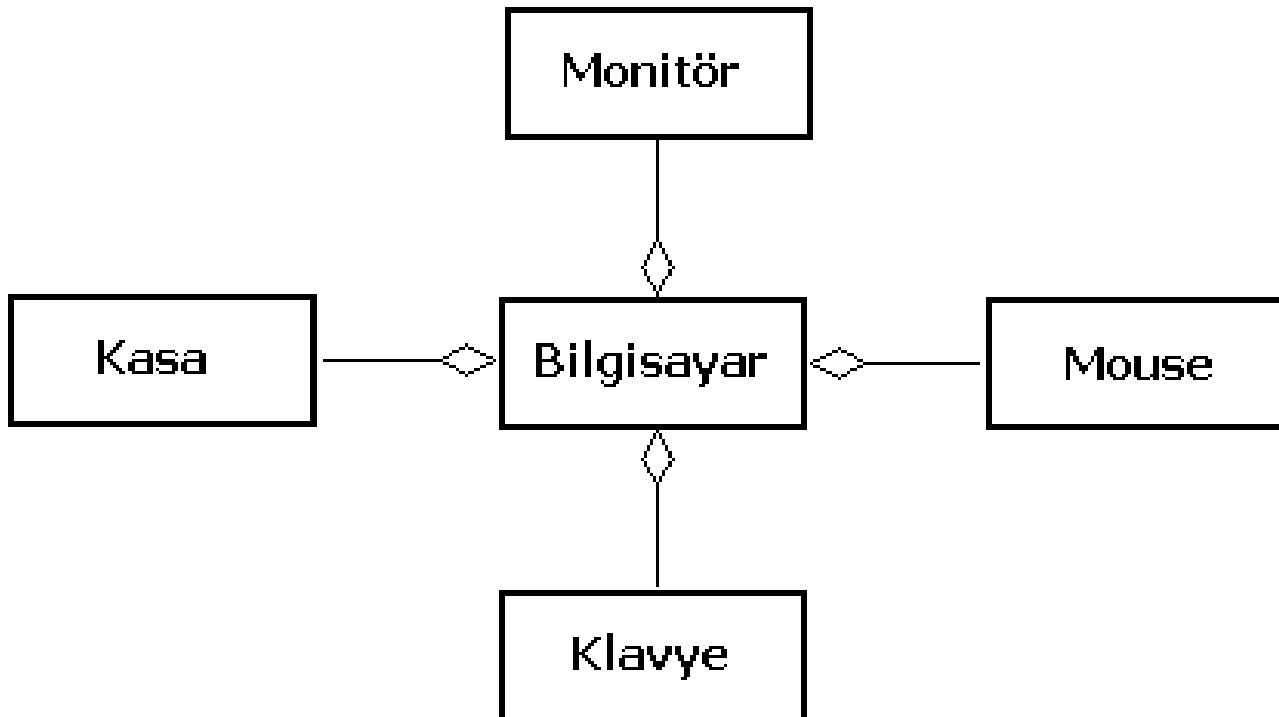
B.)Kümeleme(Aggregation) ve Birleştirme(Composition)

- ∞ Kümeleme ve birleştirme, sınıflar arasındaki parça-bütün ilişkileridir.
- ∞ **Kümeleme**de (Aggregation) alt parçalar, sadece o sınıfa ait üyeler değildir, kendi başlarına da kullanılabilirler. UML diyagramlarında bağlantı çizgisinin ucuna içi boş bir dörtgen çizilerek gösterilir.



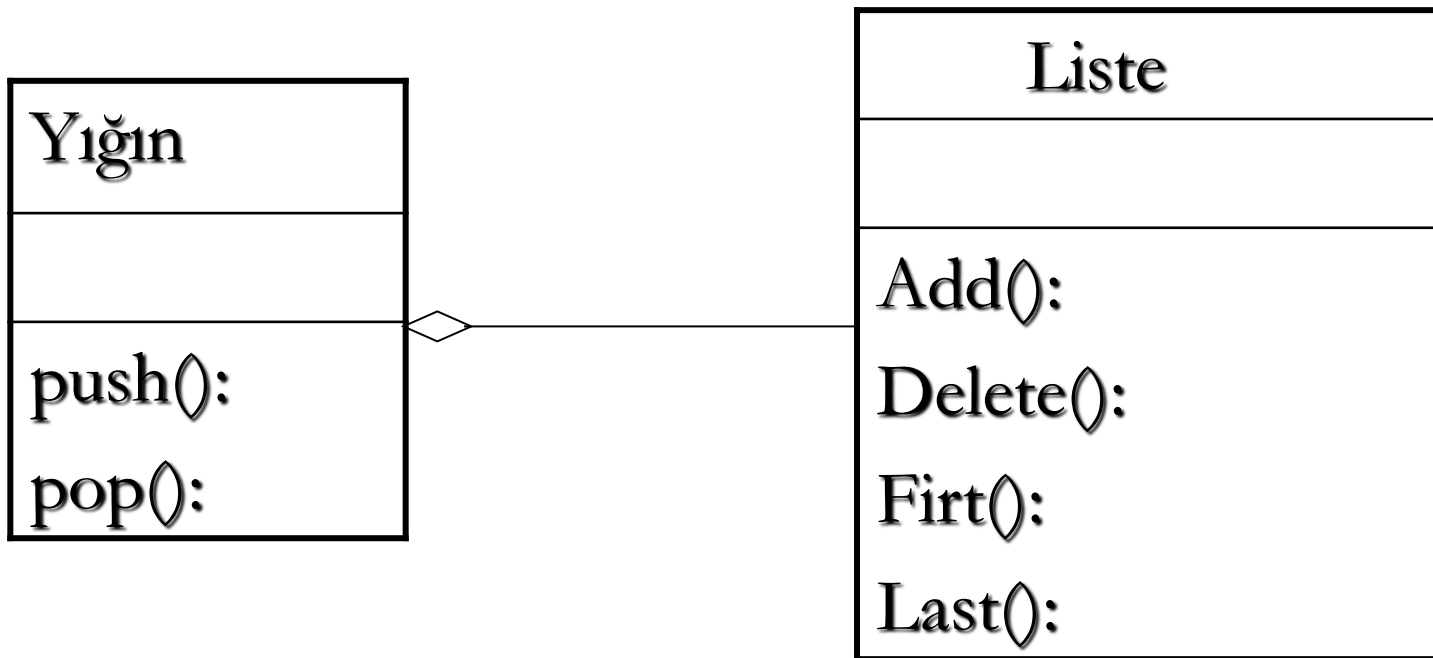
Aggregation İlişkisi

- ☞ Bağımsız iyelik; Nesneler arasındaki bir tür parça bütün ilişkisidir. Bilindiği gibi günlük hayatta karşılaşılan pek çok komplike varlık, birden fazla alt parçanın birleşiminden oluşmaktadır.
- ☞ **Aggregation (Kümeleme)** ilişkisi; bu tür varlıklar ile onları oluşturan parçaların modellenmesini sağlar.





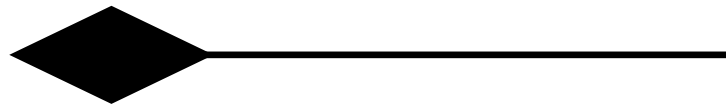
Kümeleme İlişkileri



∞ “Liste” sınıfı, “Yığın” sınıfına bağlı olmadan da yaratılabilir ve kullanılabilir.

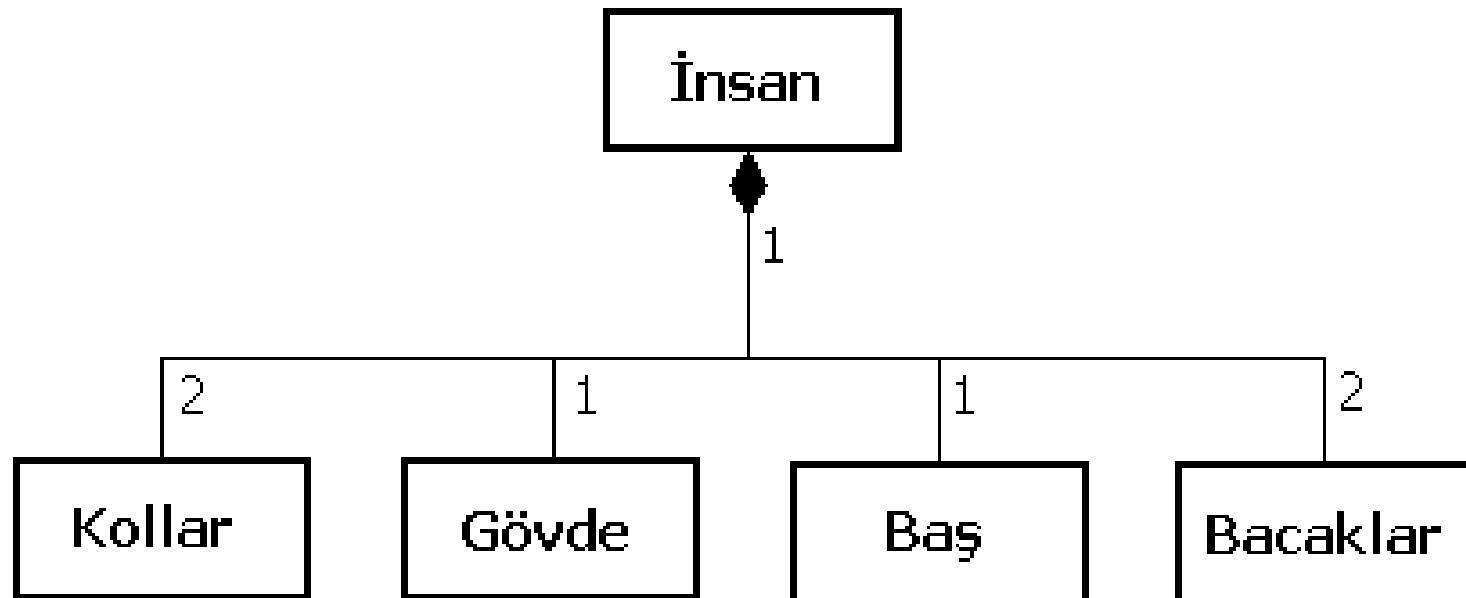
Composition İlişkisi

- ∞ Birleştirmede (Composition) ise alt parçalar, o nesneyi meydana getirmek için oluşturulurlar ve kendi başına kullanılamazlar.
- ∞ UML gösterimi, bağlantı çizgisinin ucuna çizilen içi dolu bir dörtgendir.

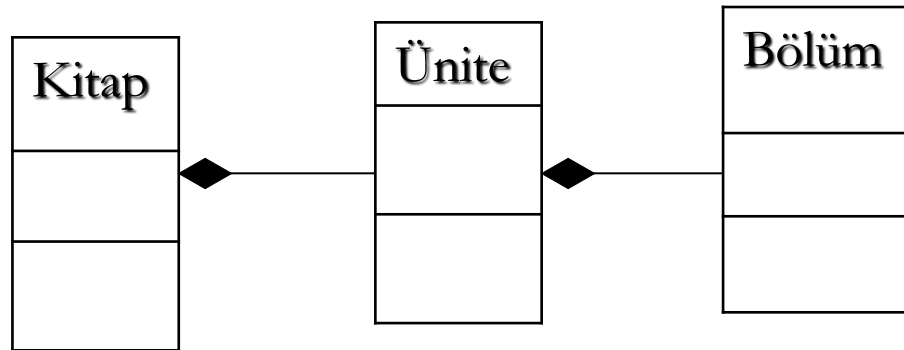


Composition İlişkisi

- ☞ Composite Aggregation : Bağımlı iyelik
- ☞ Parça ve bütün arasında organik bağımlılık vardır.



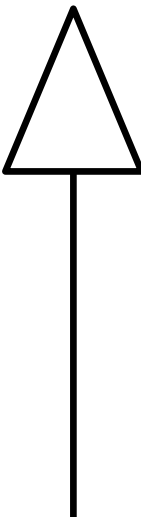
Birleştirme İlişkileri



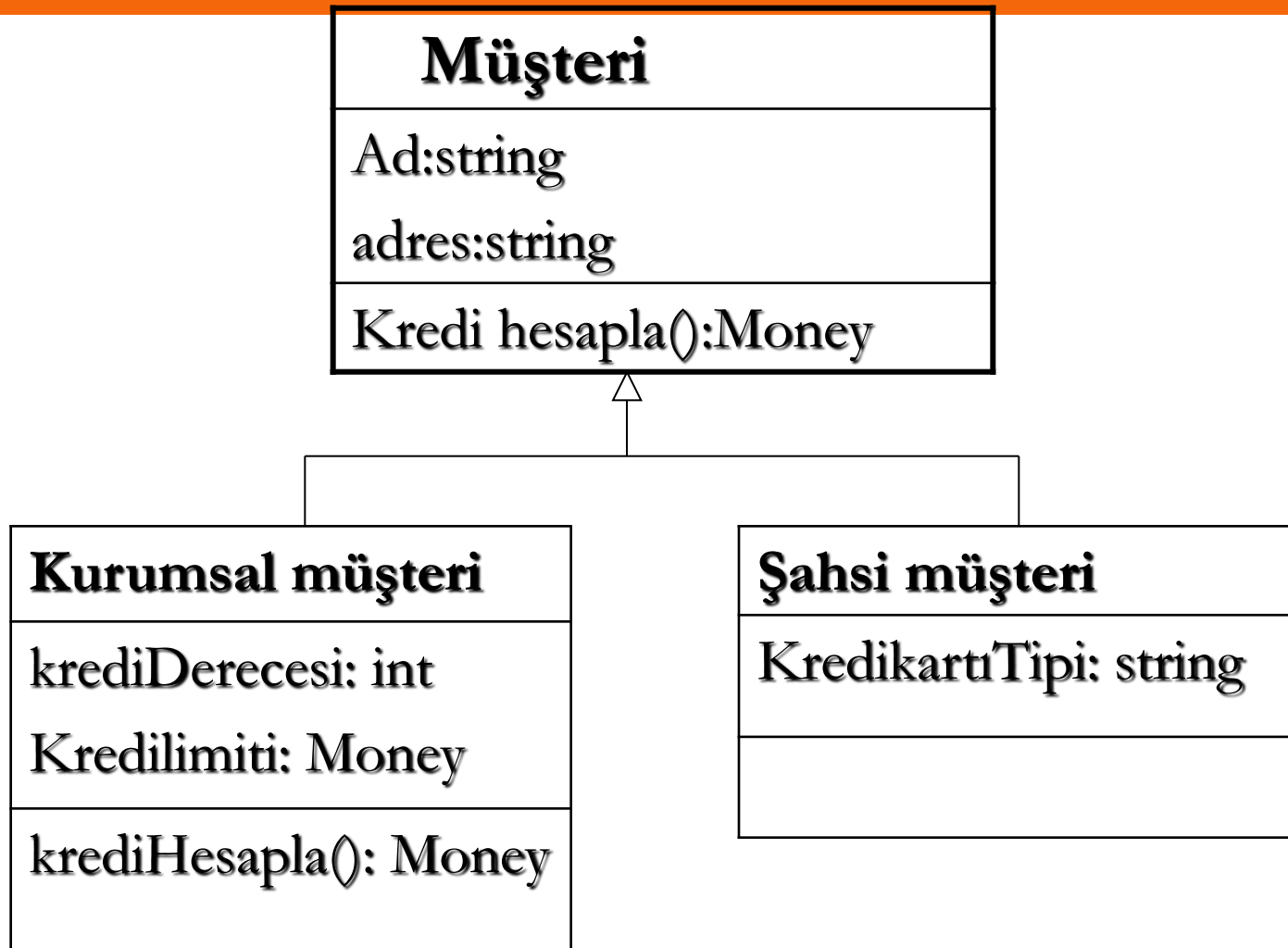
- “Ünite” nesnesi olmadan, “Bölüm” nesnesi tek başına olamaz.
- Aynı şekilde “Kitap” sınıfına bağlanmayan “Ünite” sınıfı da olamaz.
- “Kitap” nesnesi yok edildiğinde ona bağlı tüm nesnelerde yok olur.

C.)Genelleştirme (Generalization)

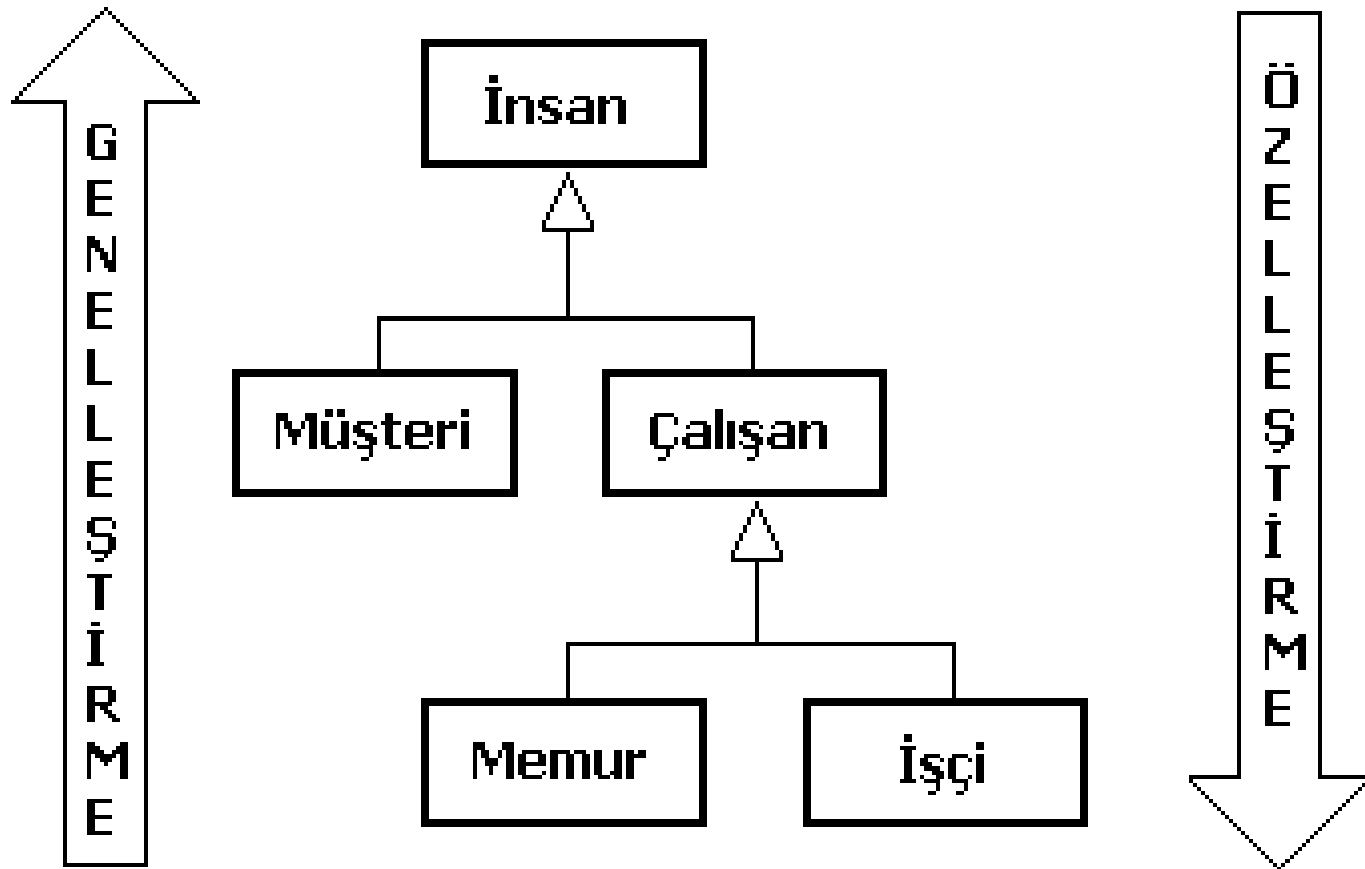
- ☞ Genel tanımlar içeren ana bir sınıf ile daha özel ayrıntılara sahip alt sınıflar arasındaki ilişki türüne **genelleştirme** denir.
- ☞ Alt sınıf, üst sınıftan türetilir.
- ☞ Üst sınıfın özellikleri alt sınıfta aynen kullanılabilir.
- ☞ Böylece tanımlanmış olan özelliklerin tekrar tanımlanmaları gerekmez.
- ☞ Tasarlanan sınıfların tekrar kullanılabilirliği artar.
- ☞ UML' de genelleştirme, bağlantı çizgilerinin ucuna içi boş bir üçgen çizilerek gösterilir.



Genelleştirme



Genelleme (Generalization)



D.)Çok Şekillilik ve Soyut Sınıf

- ☞ Çok şekillilik (Polymorphism), farklı sınıflardan yaratılmış olan nesnelerin, aynı mesaja farklı tepkiler vermesi olarak tanımlanabilir.
- ☞ Alt sınıfın miras aldığı bir işlem aynen kullanılabilir.
- ☞ Örneğin “Müşteri” sınıfında tanımlı kredihesapla() metodu, “ŞahsiMüşteri” alt sınıfında kullanılır. Ayrıca “KurumsalMüşteri” sınıfında “kredihesapla()” işleminin ise çok şekilli olduğu söylenebilir.

Soyut sınıf

- ☞ Soyut (Abstract) sınıf, üzerinden doğrudan nesne tanımlayan sınıftır.
- ☞ Soyut sınıfın UML tanımı, sınıftan ya da soyut işlemiden önce yazılan `<<abstract>>` anahtar kelimesidir.

- ☞ Bir sınıf içerisinde tanımlı olan bir işlem aynı isimle ancak farklı tip veya sayıda değişik parametreler ile tanımlanabilir. Buna **aşırı yükleme (overloading)** denir.
- ☞ Örneğin toplama işlemine aldığı parametrelere göre birçok farklı şekilde tanımlanır
 - Integer topla(integer a, integer b)
 - Float topla(integer a, float b)
 - Float topla(float a, integer b)
 - Float topla(float a, float b)

NESNEYE YÖNELİK SİSTEM ANALİZİ

Yazılım geliştirme yaşam döngüsü açısından nesneye yönelik sistem analizi, geleneksel yapısal yaklaşımla aynı aşamalara sahiptir.

- ∞ Planlama, analiz, tasarım, gerçekleştirme aşamaları ve aşamalarda gerçekleştirilen aktiviteler de aynıdır.
- ∞ Yapısal sistem geliştirmede işlemlerin tanımlanması ve modellenmesi üstünde durulurken, nesneye yönelik yaklaşım nesneler ve etkileşimleri vurgular.
- ∞ Her iki yaklaşım geliştirme aşamalarında modelleme üstünde dursalar da, değişik açılardan ele alırlar. Ör; sistem analisti, veri akış diyagramları yerine, etkileşim diyagramları kullanır.

- ✎ Bundan başka, geleneksel yaklaşımda kullanılan varlık-ilişki diyagramları ile nesneye yönelik yaklaşımdaki sınıf diyagramları, farklı gösterimler kullanmalarına karşın birçok ortak kavram içerirler.
- ✎ İki yaklaşım arasındaki temel fark, sistem aktivitelerini göstermek için kullanılan diyagramların farklılığı ve çeşitliliğidir.

Nesneye yönelik yaklaşımda uygulamanın gereksinimlerini tanımlamak için birbiriyle ilişkili ancak bağımsız iki diyagramdan yararlanılır:

1)Sınıf (Class Diagram) diyagramı; Bir sistemdeki tüm nesnelerin sınıf tanımlarını ve ilişkilerini göstermek için kullanılır. Aynı zamanda her sınıfın özellikleri ve işlemleri de bu diyagramda belirtilir.

2)Kullanım senaryosu (Use Case Diagram) diyagramları; yeni sistemdeki kullanımların, senaryoların belirlenmesi için kullanılır. Genel olarak, sistemin nasıl kullanılacağını modeller. Sistemdeki olay tablosunun bir özet gösterimi şeklindedir. Bu sayede sistemin gerçekleştirmesi gereken fonksiyonlar belirlenmiş olur. Tek, anlaşılabilir bir kullanım senaryosu diyagramı ile sistem modelleneneceği gibi, daha küçük diyagramlardan oluşmuş bir kullanım senaryosu modeli de oluşturulabilir.

Tasarlanacak olan sistem analiz edilirken temel amaç, çözülecek problemin doğru, mantıklı, anlaşılır ve test edilebilir bir modelini oluşturmaktır. Problemin çözülmesi analiz aşamasında gerçekleşmez. Buradaki hedef, problemin tam olarak anlaşılmasıdır. Yazılım nesneleri gibi tasarıma yönelik detaylarla bu aşamada ilgilenilmez.

Kullanım Senaryosu Modellemesi (Use-Case Modeling)

- ∞ Bir sistemin davranışı dış olaylara verdiği tepkilerdir.
- ∞ UML'de, sistemin dışarıdan görülebilen ve test edilebilen davranışları, kullanım senaryoları ile gösterilir.
- ∞ **Senaryolar:** Anlamalı bir sonuca ulaşmak için **aktör** ile sistem arasında gerçekleşen olaylardır.
 - **Aktör** ise bir senaryoya etkileşime geçen kimse ya da şey, kısaca sistemin kullanıcısıdır. Bir insan veya başka bir sistem olabilir.
 - Aktör, senaryodan kullanabileceği bir sonuç bekler.

KULLANIM SENARYOLARI (Use Case)

- ☞ Kullanım senaryosu modellemesi kolay anlaşılırdır, müşteri ile iletişimi kolaylaştırır.
- ☞ Gerçeklenen sistemin isteklere uygun olup olmadığı senaryolar ile test edilebilir.
- ☞ Nesneye yönelik analiz için uygun bir başlangıç oluşturulur.

KULLANIM SENARYOLARI (Use Case)

- ∞ **Aktörler:** Senaryo ya da aktörlerden ilk olarak hangisinin belirleneceği sistem analistinin vereceği karardır.
- ∞ Aktörler UML'de kullanım senaryosu diyagramları çizilirken **çöp adam**larla ifade edilir.
- ∞ Ancak aktör özelliklere ve işlemlere sahip olabileceğinden, normal bir sınıf şeklinde de gösterilebilir.

KULLANIM SENARYOLARI (Use Case)

Aşağıdaki örnekte güncel bir uygulama olan internette alışveriş için uygulamanın yapısı, teknik alt yapı, alışveriş sırasında gerçekleştirilen işlemler üzerinde durulmuştur. Sitenin genel yapısı:

- ☞ Site kahve satışı yapmaktadır.
- ☞ Hazır kahveler satın alınabileceği gibi, müşteri dilediği çeşitleri harmanlayarak özel kahve siparişi de verebilmektedir.
- ☞ Bir alışverişte birden fazla çeşit kahve siparişi verilebilir.

KULLANIM SENARYOLARI (Use Case)

- ☞ Siparişi tamamlamak için müşteri teslimat ve ödeme formlarını doldurur.
- ☞ Kredi kartı ve banka havalesi ile ödeme yapılabilir.
- ☞ Arka planda müşterinin bilgileri ve ödemesi doğrulanır.
- ☞ İstenen özelliklerdeki sipariş, kahve sağlayan depodan alınır, fatura basılır ve müşteriye gönderilir.

KULLANIM SENARYOLARI (Use Case)

Aktörleri ve senaryoları belirlemek için öncelikle sistemin sınırları belirlenmelidir. Bunun için ilk olarak sistemin genel akışı adım adım özetlenir;

- ☞ Müşteri, hazır bir kahve seçer. Özellikleri ile fiyatı da gösterilir.
- ☞ Müşteri istediği kahveleri harmanlayarak kendine özel bir kahve oluşturabilir. Fiyat da seçilen kahvelere göre yeniden hesaplanır.
- ☞ Müşteri 1. ve 2. adımları tekrarlayarak alışverişine devam eder.

KULLANIM SENARYOLARI (Use Case)

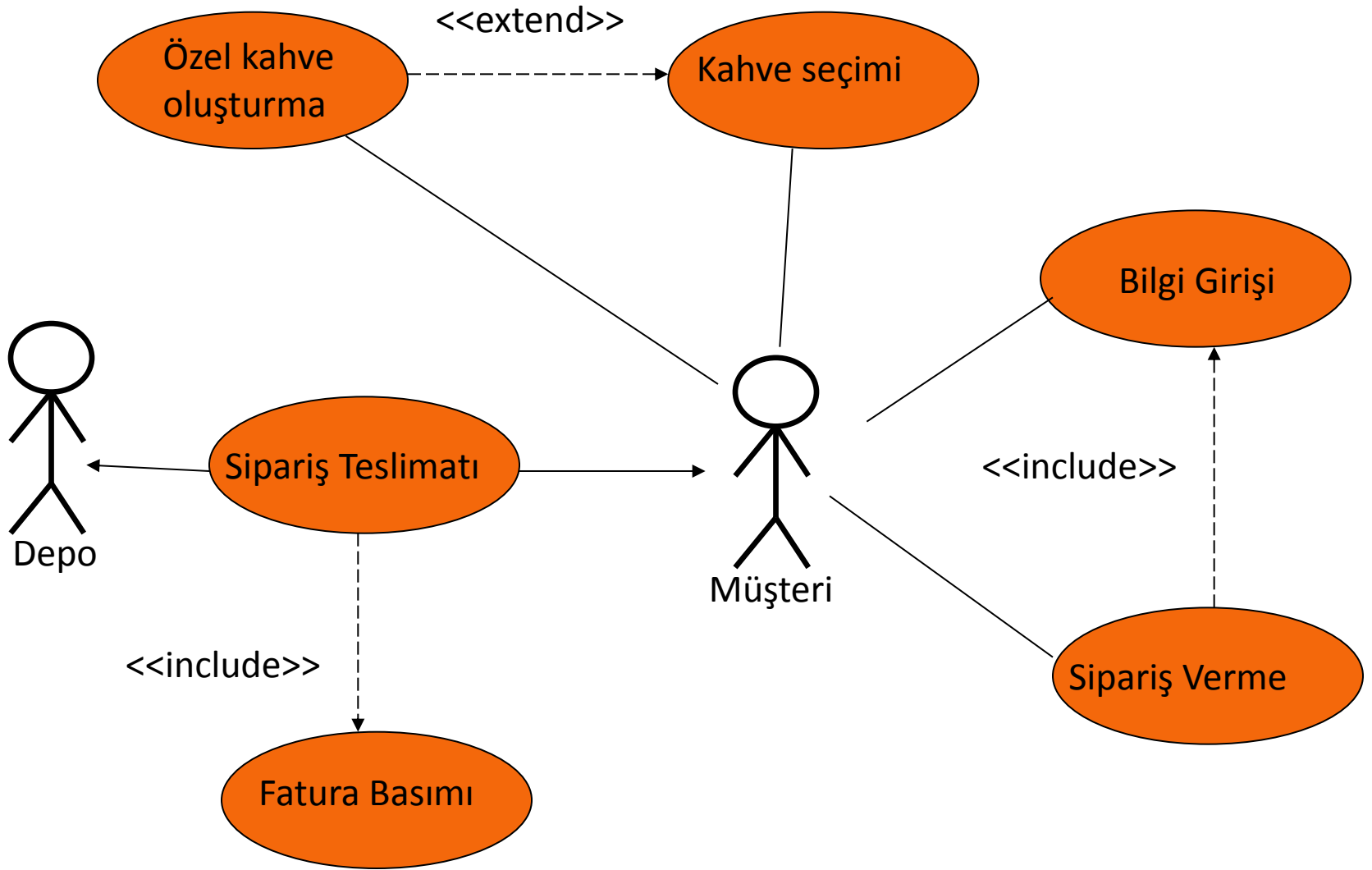
- ☞ Müşteri siparişini siteden verir. Siparişi tamamlamak için teslimat ve fatura bilgileri, ödeme detaylarını içeren bir form doldurur.
- ☞ Sipariş sisteme girilince, müşteri ve ödeme bilgileri doğrulanır.
- ☞ İstenen özelliklerdeki kahve, sağlayıcı depodan alınır, faturası basılır.
- ☞ Sipariş, fatura ile birlikte müşteriye gönderilir.

KULLANIM SENARYOLARI (Use Case)

- ✎ Senaryo, bir aktöre fonksiyonel bir değer sunan bir bütündür.
- ✎ Herhangi bir senaryoyla ilişkisi bulunmayan bir aktör sistemde gereksizdir. Ancak hiçbir aktörle ilişkide olmayan senaryolar bulunabilir.
- ✎ Kullanım senaryoları belirlenirken sistemin aktörlere karşı sorumlulukları düşünülmelidir.

KULLANIM SENARYOLARI (Use Case)

- ☞ Kullanım senaryolarının UML gösterimi elips şeklindedir.
- ☞ Senaryolarla ilişkide olan aktörler de diyagramda yer alır. Genelde aktör ve senaryoların etkileşimde olması beklenir.
- ☞ Bir aktörün direk olarak bir başka aktörle etkileşimde olması ise önerilmez. Aralarında mutlaka etkileşimi sağlayan bir senaryo bulunmalıdır.



Kahve sitesi sistemi için kullanım senaryosu diyagramı

KULLANIM SENARYOLARI (Use Case)

Kullanım senaryoları arasında 3 farklı ilişki tanımlanabilir.

- ✎ **İçerme(Include-Use):** Bir senaryoda kullanılan başka bir senaryoyu belirtir.
- ✎ **Genişletme(Extend):** Senaryolar doğal akışlarına göre hazırlanırlar. Özel durumlarda sapmalar olabilir. Bu gibi senaryolarda genişletme ilişkisi ile gösterilir.
- ✎ **Özelleştirme(Specialization):** Sınıflar arası türeme ilişkileri gibi, senaryolar da türetilebilir.

KULLANIM SENARYOLARI (Use Case)

Her senaryo bir senaryo dokümanı ile ayrıca detaylandırılmalıdır.

Senaryo dökümanının yapısı;

- ✎ Senaryo adı
- ✎ İlgili aktörler
- ✎ Senaryonun gerçekleşebilmesi için gerekli ön koşullar
- ✎ Senaryo tamamlandığında sistemin ulaşacağı durumu tanımlayan son koşullar
- ✎ Senaryonun temel akışı
- ✎ Özel durumlarda yürütülecek alternatif akışlar

“Sipariş Verme” Senaryo Dökümantasyonu

Senaryo KS1	Sipariş verme
Birinci Aktör	Müşteri
İlgililer ve Beklentileri	<i>Kahve sitesi:</i> Müşterinin siparişi doğru ve eksiksiz kaydedilmeli,ödeme ve teslimat bilgileri doğru alınmalı. <i>Depo:</i> Siparişteki kahve miktarları doğru ve eksiksiz olmalı.
Ön Koşullar	Müşteri alışveriş sitesinde olmalı,sunulan seçenekler ile istediği kahveyi oluşturmalıdır.
Son Koşullar	Ödeme ve teslimat bilgileri doğrulanmıştır.Müşterinin siparişi sistem veritabanına teslimat yapılmak üzere kaydedilmiştir.
Ana Akış	1-Müşteri sipariş etmek istediğini belirtir. 2-Müşteri teslimat için adını ve adresini,teslimat bilgilerinden farklıysa fatura bilgilerini girer. 3-Müşteri ödeme şeklini seçer ve gerekli ödeme bilgilerini girer. 4-Sistem sipariş için benzersiz bir tanımlayıcı kod üretir,müşteri hesap bilgileri ile birlikte kaydeder.
Alternatif Akış	2a-3a:Müşteri gerekli tüm bilgileri sağlamadan siparişi tamamlar. Sistem bir hata mesajı vererek eksik olan bilgilerin tamamlanmasını ister.

Kullanım Senaryolarında Sözleşmeler(Contracts)

- ∞ Kullanım senaryoları, birçok uygulamanın modellenmesinde genellikle yeterli olmaktadır.
- ∞ Ancak bazı senaryolar, karmaşık işlemler içerebilir. Bu gibi durumlarda anlaşılabilirliği arttırmak için sözleşme yazılır.
- ∞ Sözleşmeler, ön koşullar sağlandığında sistemin işlemesi ile uygulama nesnelerinin alacağı son durumları tarif eden yapılardır.

SINIF MODELLEMESİ

- ✎ Nesneye yönelik analizde sistemin iç durumu sınıf modeli ile tanımlanır.
- ✎ Bu modellemede sınıflar ile birlikte özellikleri, işlemleri, bağlantılar, bütünleşmeler, bileşimler ve genelleştirmeler de yer alır.
- ✎ Nesneye yönelik sistem analizi sırasında kullanım senaryosu modellemesi ile sınıf modellemesi paralel olarak ilerler.
- ✎ Kullanım senaryoları ile sınıfların belirlenmesi kolaylaşırken, sınıflar da gözden kaçırılan kullanım senaryolarının belirlenmesini sağlar.
- ✎ Bu iki model ile uygulama alanının modeli oluşturulmuş olur.

SINIF MODELLEMESİ

- ❧ Sınıf modellemesinde yer alan sınıflar, gerçek dünyayı oluşturan kavramsal nesnelerdir.
- ❧ Yazılım sınıfları bu aşamada düşünülmesine de, tasarıma kılavuzluk eder.
- ❧ Sınıfları belirlemek için, kullanım senaryolarında yer alan isimlerden ve isim tamlamalarından yararlanılır.
- ❧ Örnek olarak KS1 kullanım senaryosundaki isimler ve tamlamaları verilmiştir.
 - Müşteri sipariş etmek istediğini belirtir.
 - Müşteri teslimat için adını ve adresini, fatura bilgilerini girer.
 - Müşteri ödeme şeklini seçer ve ödeme bilgilerini girer.
 - Sistem benzersiz tanımlayıcı kod üretir, müşteri hesap bilgileri ile birlikte veritabanına kaydeder.

SINIF MODELLEMESİ

- ✎ Gereksinimlerde yer alan bir kavramın gerçekte bir sınıf olarak gerçekleştirilip gerçekleştirilmeyeceği, birkaç soru ile kesin olarak belirlenebilir:
 - Kavramla ilgili veri saklanması gerekiyor mu?
 - Değişik değerler alabilecek farklı özellikleri var mı?
 - Kavramdan birçok nesne türeyebilir mi?
 - Uygulamanın kapsama alanı içinde mi?
 - Kavramın sınırları iyi çizilmiş mi, tanımı tam yapılabiliyor mu?
- ✎ Problemin çözümü tasarım sürecine bırakılır.

Özelliklerin Belirlenmesi

- ✎ Bir sınıfın özellikleri, yaratılan nesnelere özgü değerler alan verilerdir.
- ✎ Özellikler genellikle basit veri tiplerinden oluşur.
- ✎ Birden fazla alandan oluşan, üzerinde işlem yapılan, kendi niteliklerine sahip olan kavramlar, bir sınıfın özelliği değil ayrı bir sınıf olarak ifade edilmelidir.

Bağlantıların Belirlenmesi

- ✎ Bağlantılar nesnelerin işbirliği ilişkilerinin çıkarılmasına yardımcı olmaktadır.
- ✎ Sistem gerçekleştirildiğinde sınıflar arasındaki bağlantılar, bu sınıflardaki belirli özellikler ile temsil edilir.
- ✎ Analiz aşamasında ise, bağlantılar çizgiler ve çoğullama simgeleri ile gösterilir.

- ✎ Bağlantıların belirlenmesi için, sınıfların belirlenmesinde olduğu gibi kullanım senaryolarından yararlanılır.
- ✎ Senaryolarda yer alan fiiller, olası bağlantılardır.
- ✎ Sistemin amacı dışındaki bağlantılar gereksizdir.
- ✎ Sistemin tasarımı ile ilgili bağlantılar dikkate alınmalıdır.
- ✎ Ayrıca senaryoda yer alan faaliyetler bağlantı değil etkileşimi ifade eder. Bu yüzden bağlantı ilişkiyi temsil etmelidir.
- ✎ İki sınıf arasında birden fazla yol varsa, genellikle çözümde fazlalık ilişki vardır.
- ✎ Gereğinden fazla bağlantı sınıf modellemesinin anlaşılabilirliğini azaltır.

Use Case'ler & Sınıf Tasarımı

Kullanıcı listeden seçtiği ürünün ID bilgisini kullanarak bir sipariş formu doldurur.

Sipariş formuna girilmesi gereken bilgiler; müşteri ismi, soyisim, adres, teslimat tarihi ve ödeme türü bilgisidir.

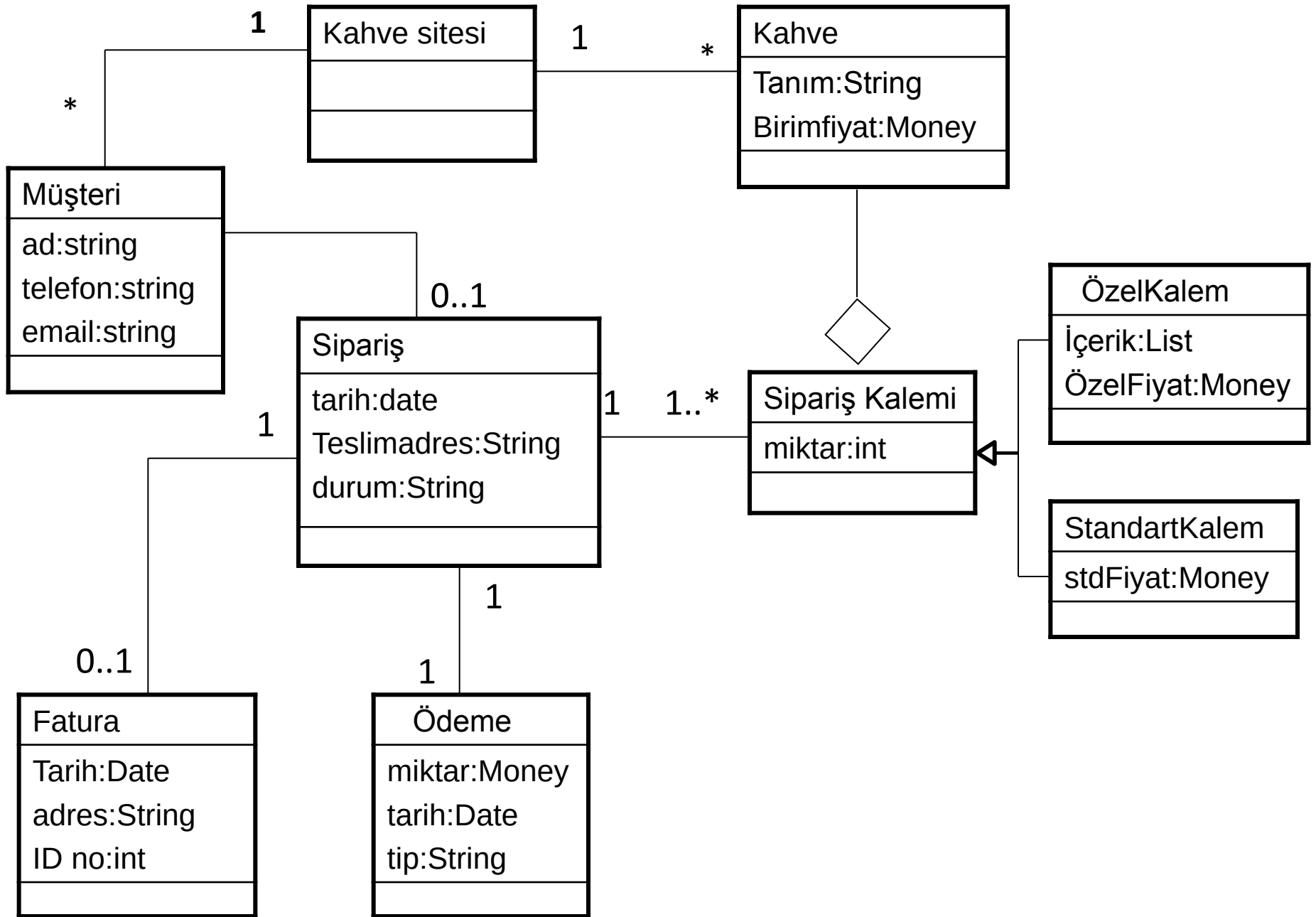
Müşteri nakit ödemeyi değil de, kredi kartı ile ödemeyi seçerse; sipariş formuna kredi kartı numarasını da girmek durumundadır.

Söz konusu sipariş bilgileri, veri tabanındaki ilgili tablolara kaydedilir.

Sınıf	Musteri, Urun, Siparis
Nitelik	İsim, Soyisim, Adres, Teslimat Tarihi, Ödeme Türü, Kart Numarası
Metot	SiparisVer

Sınıf Diyagramı

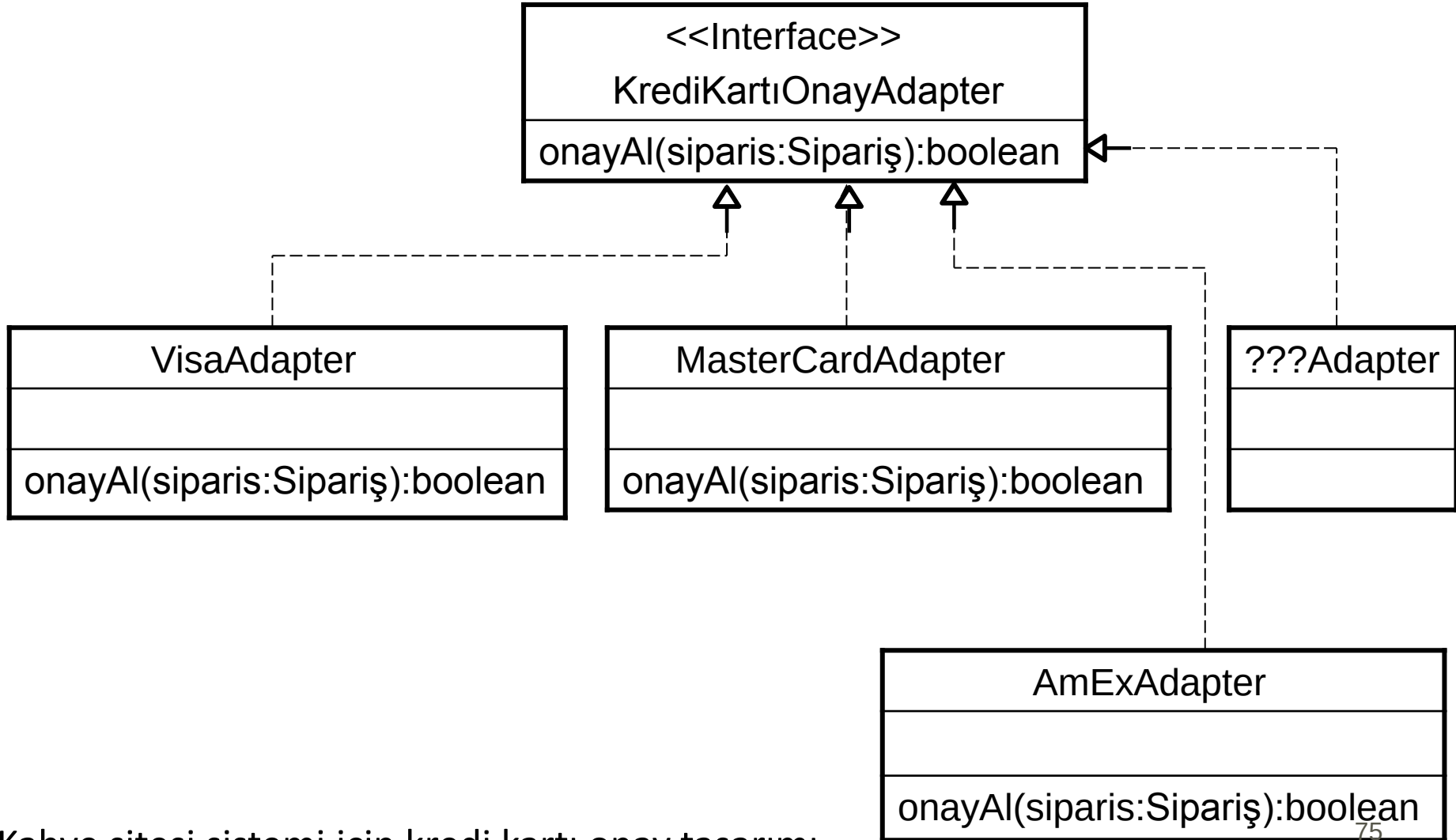
- ∞ **Sınıf diyagramı**, kullanım senaryoları ile analizi yapılan sistemin uygulama modelini gösterir.
- ∞ Bu diyagramda kavramsal sınıflar, aralarındaki ilişkiler ve sınıfların nitelikleri yer alır.
- ∞ Sınıf diyagramlarında henüz işlemler bulunmamaktadır.
- ∞ İşlemlerin belirlenmesi, sınıfların sorumluluklarının belirlenmesi demektir ve nesneye yönelik sistem tasarım safhasını oluşturur.



Şekil-11.3:Kahve sitesi sistemi için sınıf diyagramı

NESNEYE YÖNELİK SİSTEM TASARIMI

- ☞ Tasarım aşamasında analizi yapılarak modellenen sistemin mantıksal çözümü oluşturulur.
- ☞ Kavramsal sınıflar baz alınarak yazılım **sınıfları** belirlenir. Asıl önemli olan, sınıfların aralarındaki **etkileşimlerin** belirlenmesi ile nesnelere sorumlulukların atanmasıdır.
- ☞ Sorumlulukları yerine getirmek üzere sınıflarda **metotlar** oluşturulur.
- ☞ Nesneye yönelik sistem tasarımı sırasında nesnelerin birlikte çalışmalarını göstermek amacıyla **etkileşim diyagramları** çizilir.
- ☞ Bunlar **işbirliği (collaboration)** ve **ardışıklık (sequence)** diyagramlarıdır.
- ☞ Bunlarla birlikte sistemin son durumunun tasarım **sınıf diyagramı** da çizilir



Kahve sitesi sistemi için kredi kartı onay tasarımı

Etkileşim Diyagramları

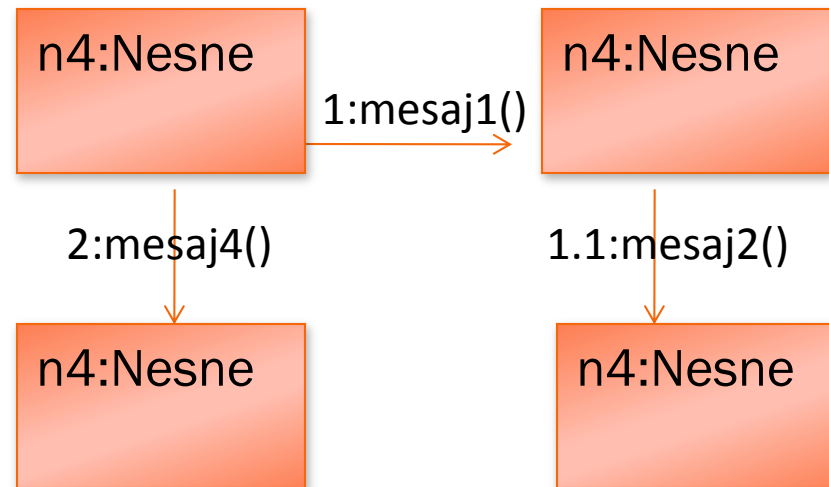
Nesneye yönelik tasarım, senaryoları gerçeklemek üzere sorumlulukların ilgili birimlere atanması ve nesneler arası işbirliği oluşturulmasıdır. Etkileşim modellenmesi iki diyagramla yapılır;

İşbirliği (Collaboration) Diyagramı: Hangi nesnelerin birlikte çalışacağını, nasıl mesajlaşacağını, bir durumu yada fonksiyonu nasıl gerçekleştireceklerini belirtir. Kullanım senaryosu diyagramında belirtilen tüm durumlar için, ayrı işbirliği diyagramları çizmek gerekir.

Ardışıklık (Sequence) Diyagramı: İşbirliği diyagramında verilen bilginin daha detaylı açıklamalarından oluşur. Nesneler arasındaki mesajlaşmaların sırası, diyagramdaki diziliş sırası ile betimlenir.

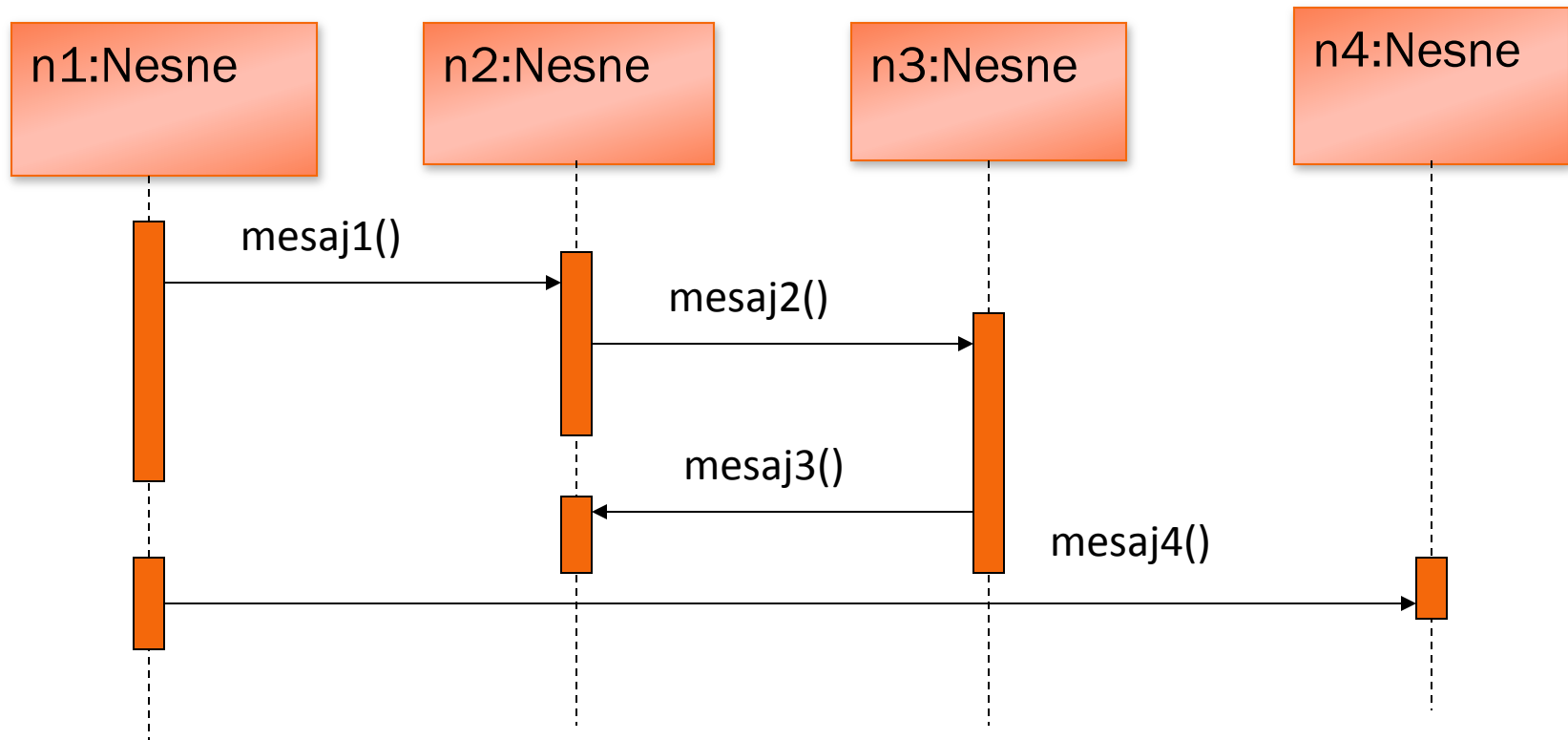
Mesaj Sıra Numaraları

İşbirliği diyagramında (Collaboration Diagram) mesajlar, yollandıkları sıraya göre numaralandırılırlar. Bir mesajın yollanması sebebiyle yollanan mesajlara, bağlı alt numaralar verilir.



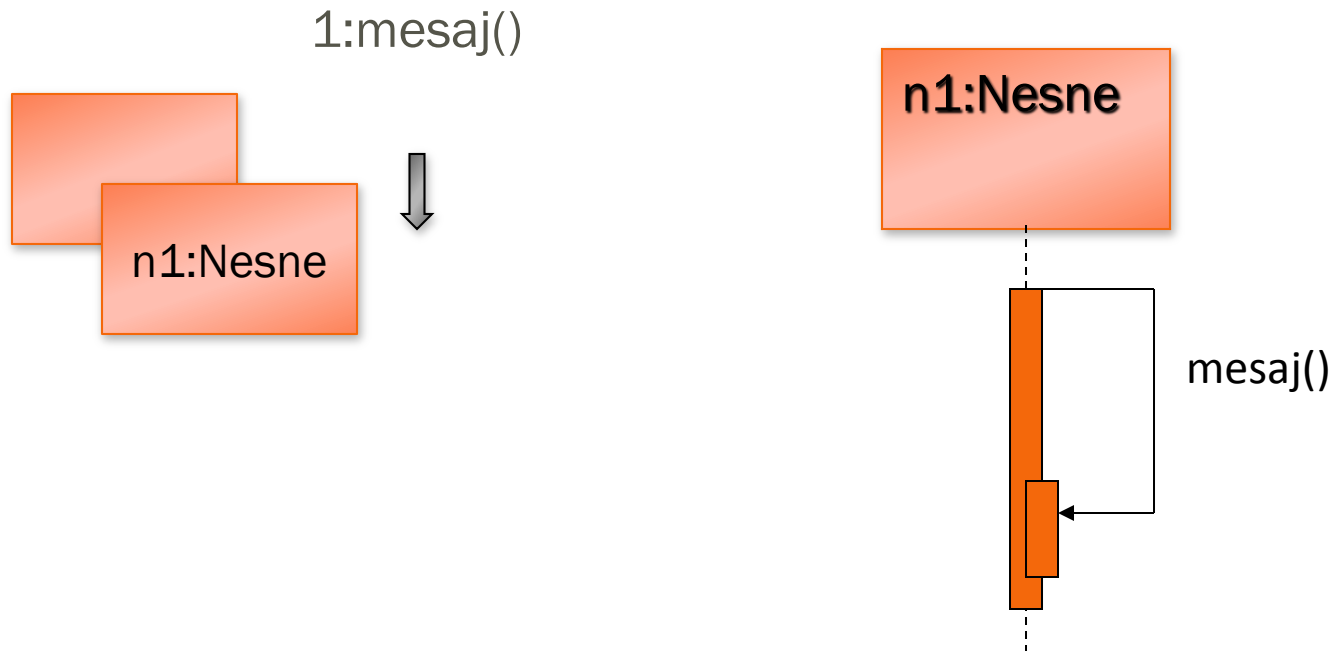
İşbirliği diyagramında mesaj sıra numaraları

Ardışılık diyagramlarında (Sequence Diagram) sıra numarası kullanılmaz. Mesajlar yollandıkları sırada alt alta çizilir.



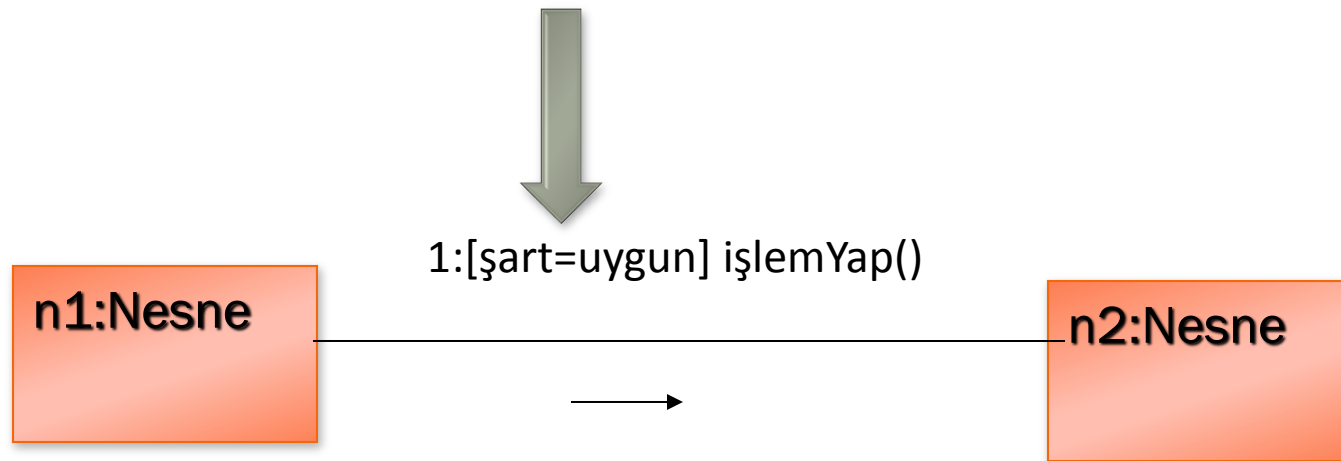
Kendine Mesaj

Tasarımda bir sorumluluğu yerine getirmek üzere nesne kendi metotlarından birini çağırabilir. Bir nesnenin kendi metotlarını çağırmasına kendine mesaj denir.

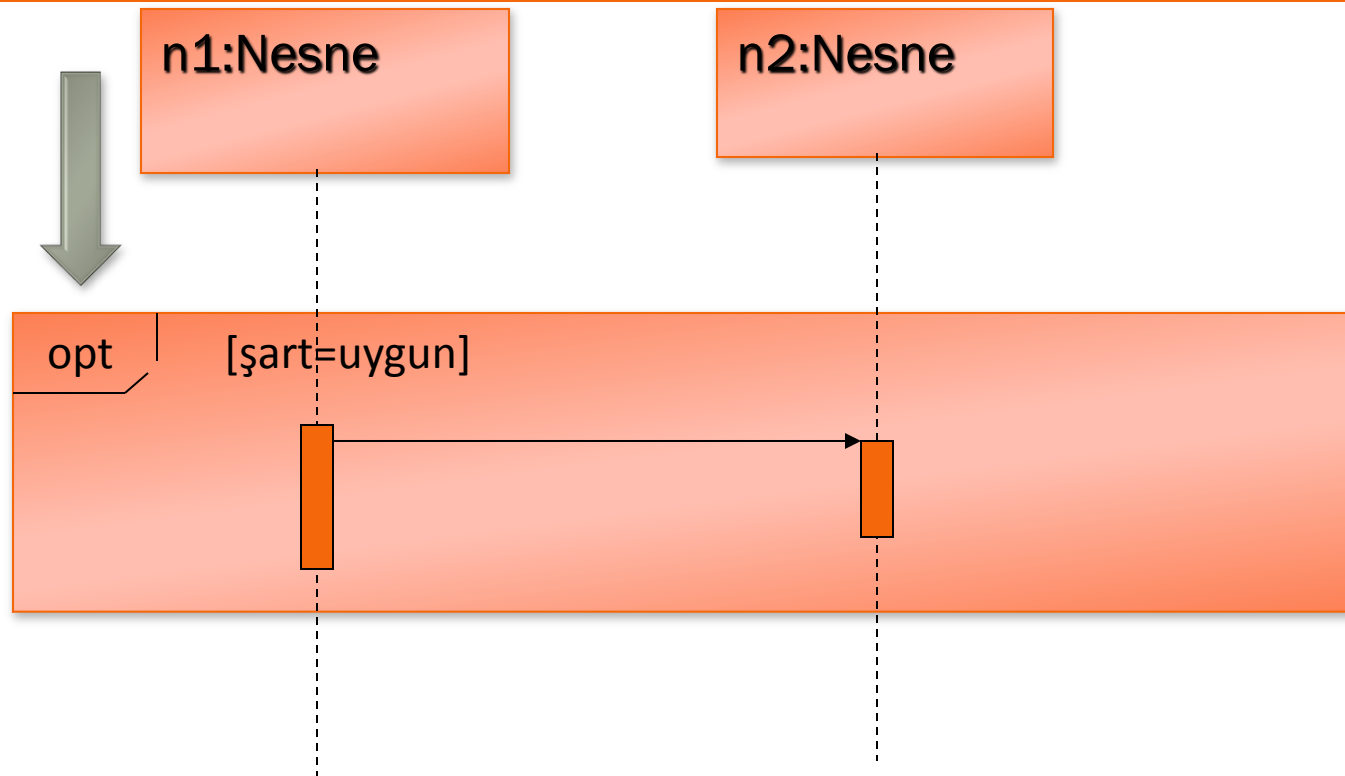


Koşullu Mesajlar

Bir mesajın sadece belirli bir koşul gerçekleştiğinde gönderilmesi gerekebilir. İşbirliği diyagramında bu durum mesajın başına eklenen şart ile gösterilir.



Ardışıklık diyagramında ise “opt” çerçevesi kullanılır.

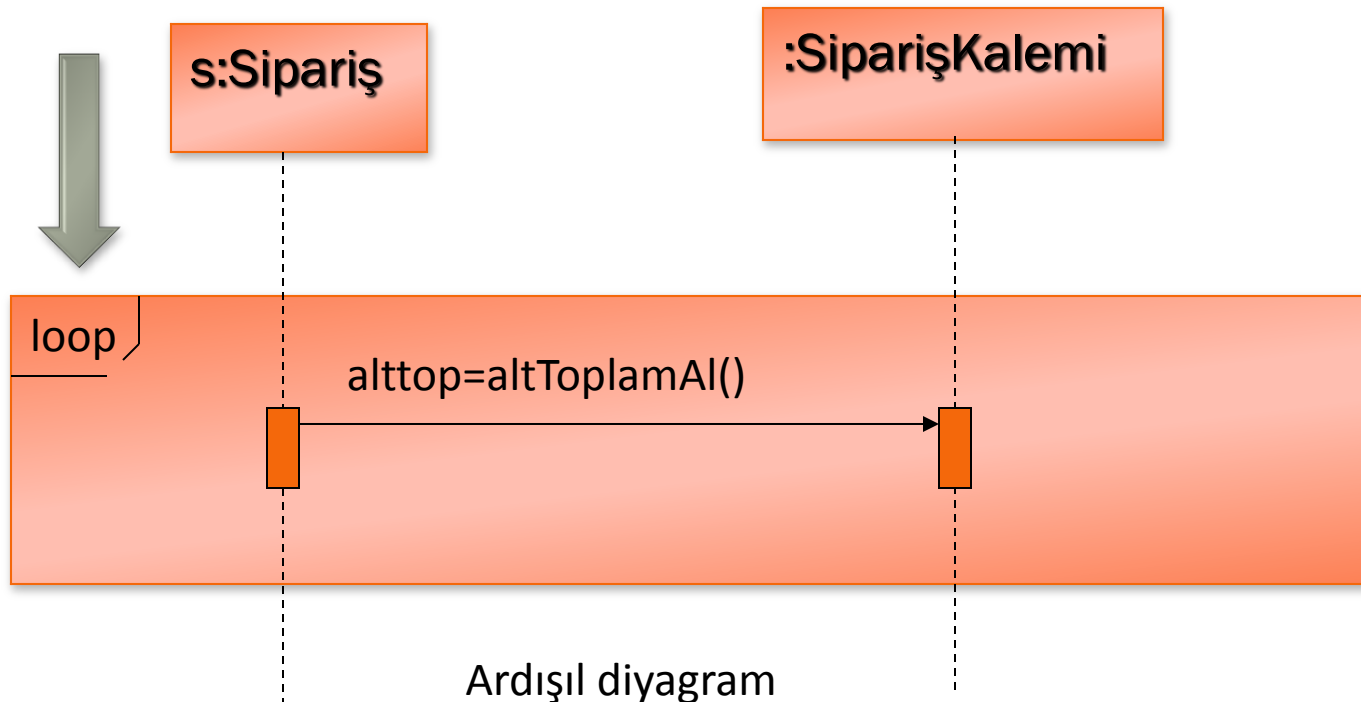


Döngüler

Bir işlemin yapılması için bir nesne grubuna mesaj gönderilmesi gerekebilir. Bir nesne grubuna yollanan mesaj yada mesajlar işbirliği diyagramında “*” simgesiyle, ardışıklık diyagramında ise “loop” çerçevesi içinde gösterilir.

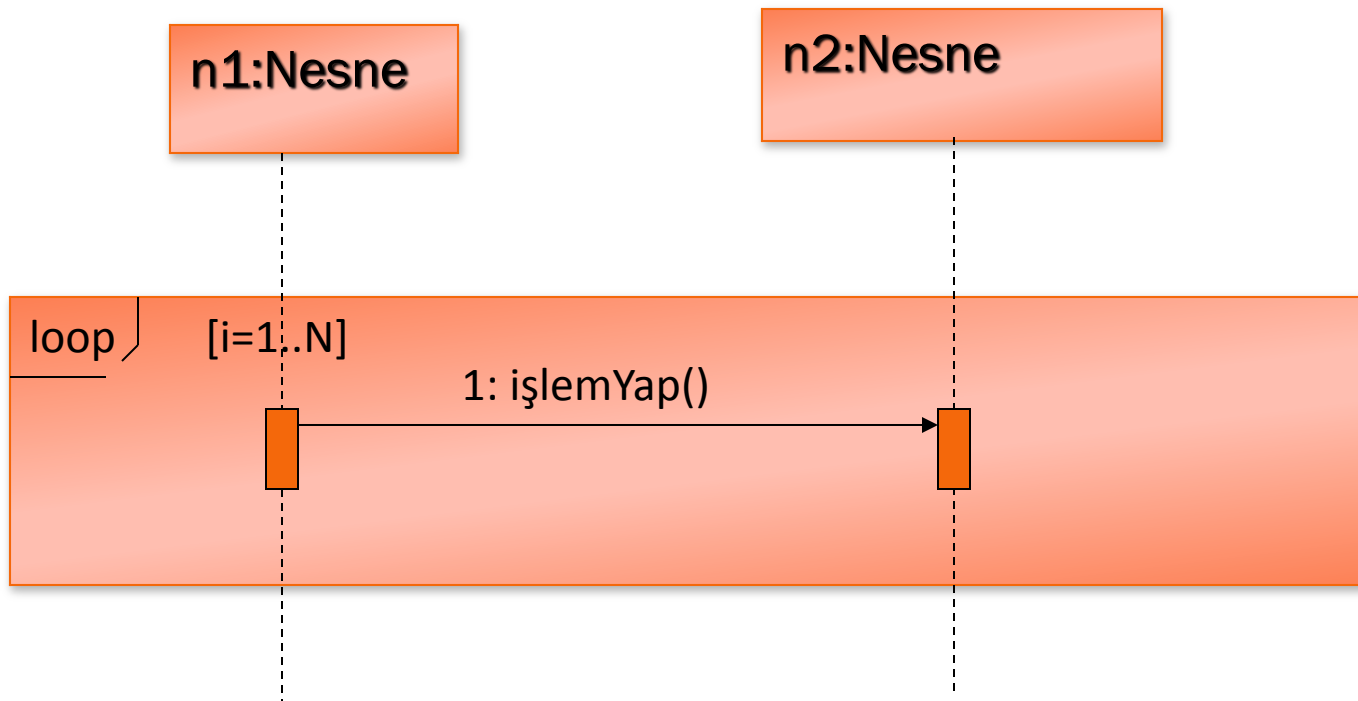
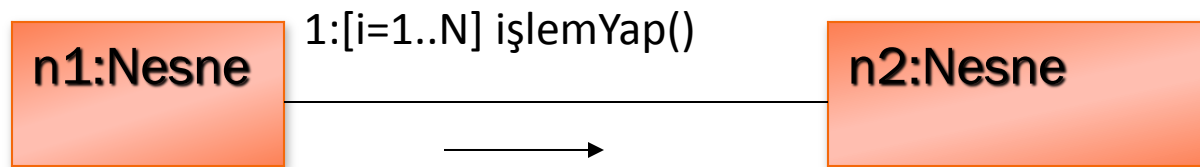


İşbirliği diyagramı



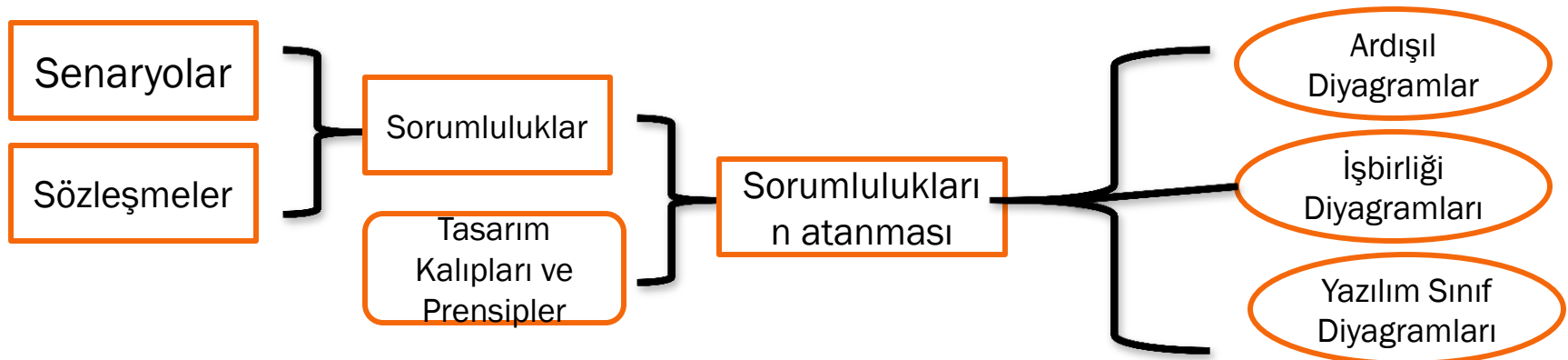
Ardışıl diyagram

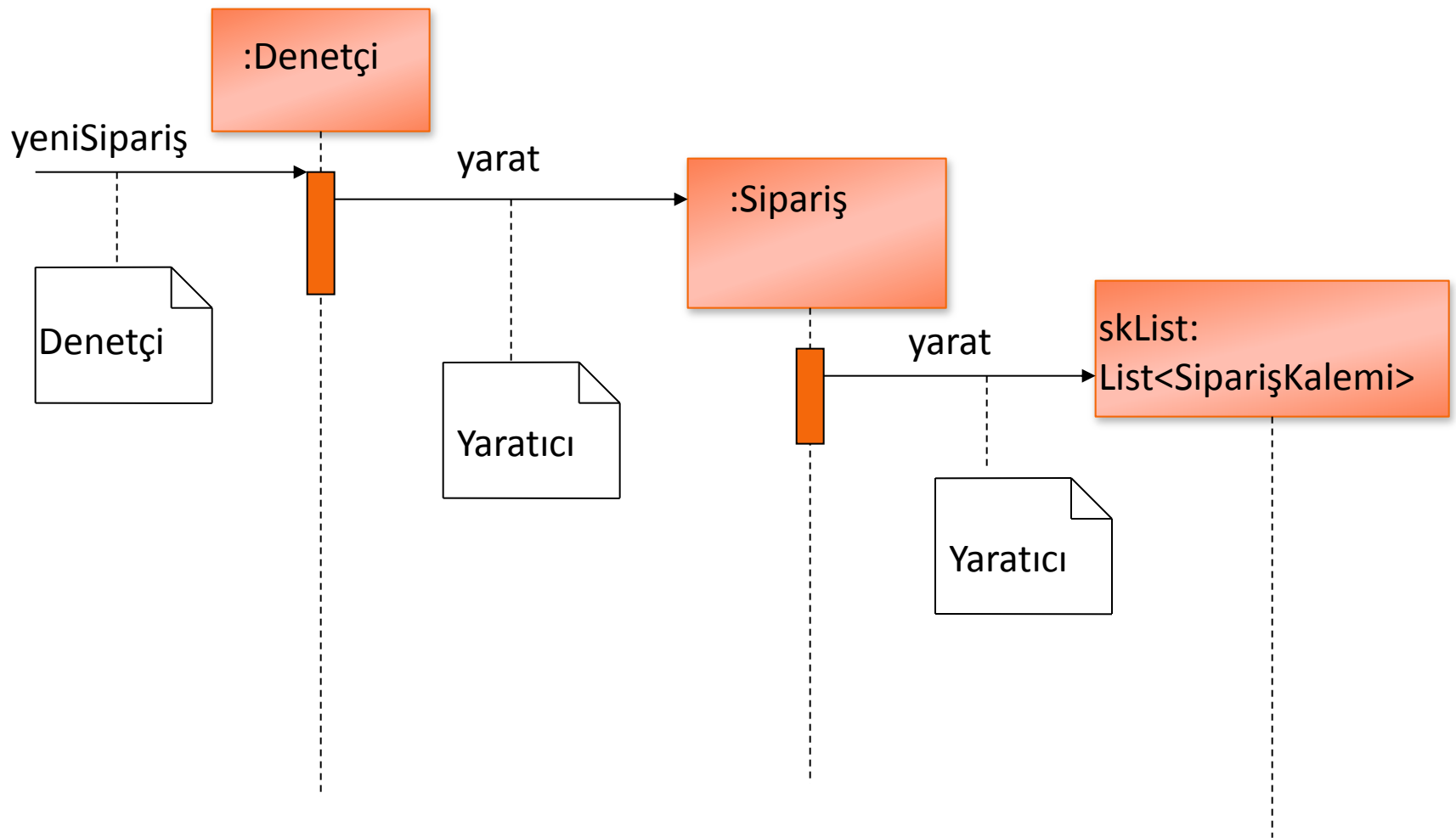
Döngü, koşulu istenirse koşullu mesajlarda olduğu gibi belirtilebilir.

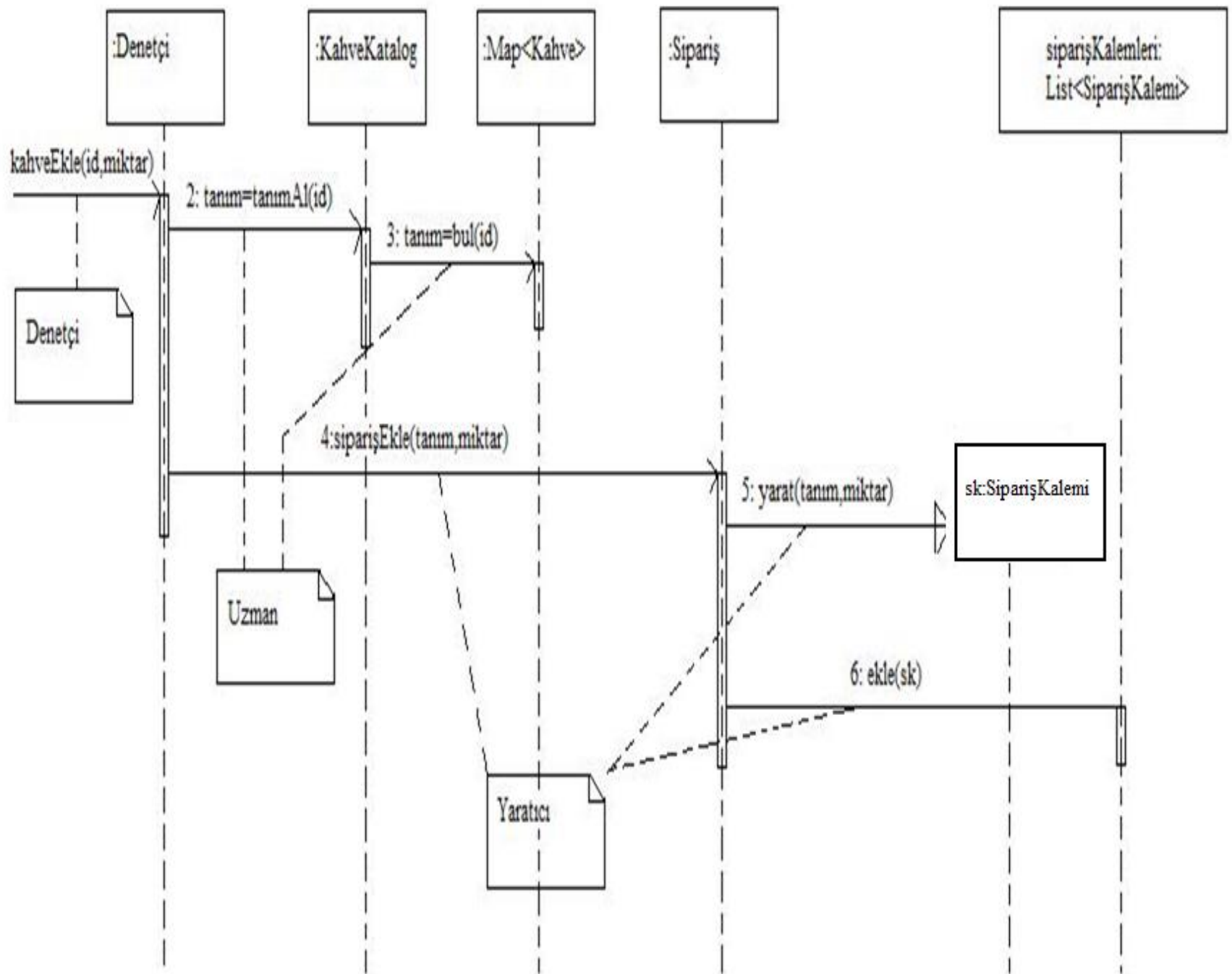


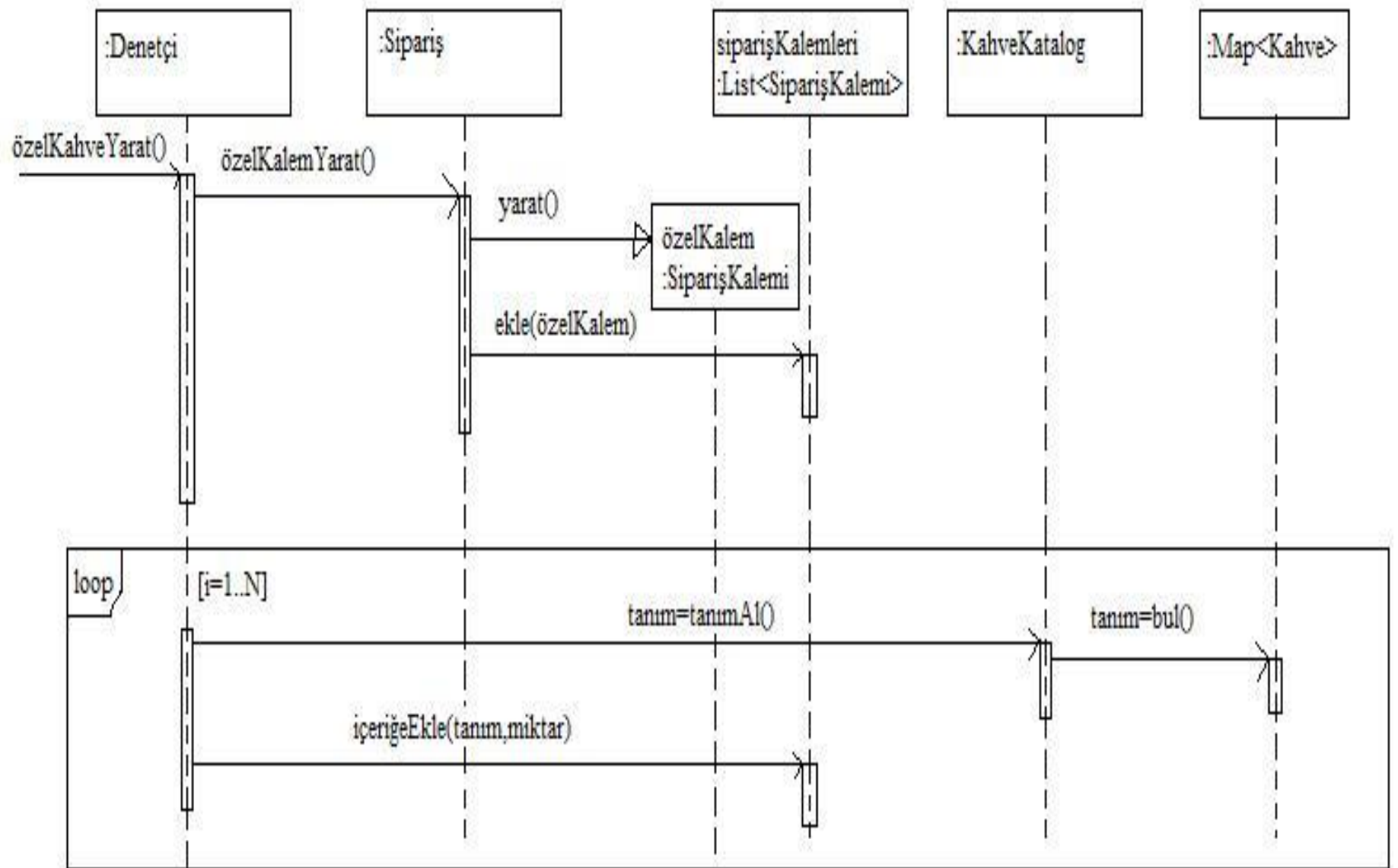
Kullanım Senaryolarının Gerçeklenmesi

- ❧ Tasarımcının bir kullanım senaryosunda yer alan durumları modelleyerek anlatması ile senaryolar gerçekleşir.
- ❧ Tasarımdaki en önemli girdi gerçekleştirilecek olan senaryolar ve varsa bunlar için yazılan sözleşmelerdir.
- ❧ Senaryoların gerçekleşmesi ile işbirliği içinde çalışan nesnelere sorumluluklar atanır.
- ❧ Senaryoların gerçekleşmesi adımıyla kullanılan girdiler aşağıda görülmektedir.









Başlangıç İşlemleri

- ✎ Başlangıç nesnesi tüm programın çalışmasını denetleyen bir nesnedir. Başlangıç nesnesi yaratıldığında ve başlangıçla ilgili diğer işlemler de yerine getirildiğinde sistem harekete geçer.
- ✎ Başlangıç sorumluluklarına ana program ya da sistemin ara yüz nesneleri de sahip olabilir. Sistemin kullanıcı arayüzü ile olan ilişkisi de Denetçi nesnesi üzerinden sağlanır. Bu sayede tasarlanan sistemin kullanıcı arayüzünden bağımsız olması da sağlanır.

Görünürlük Tipleri

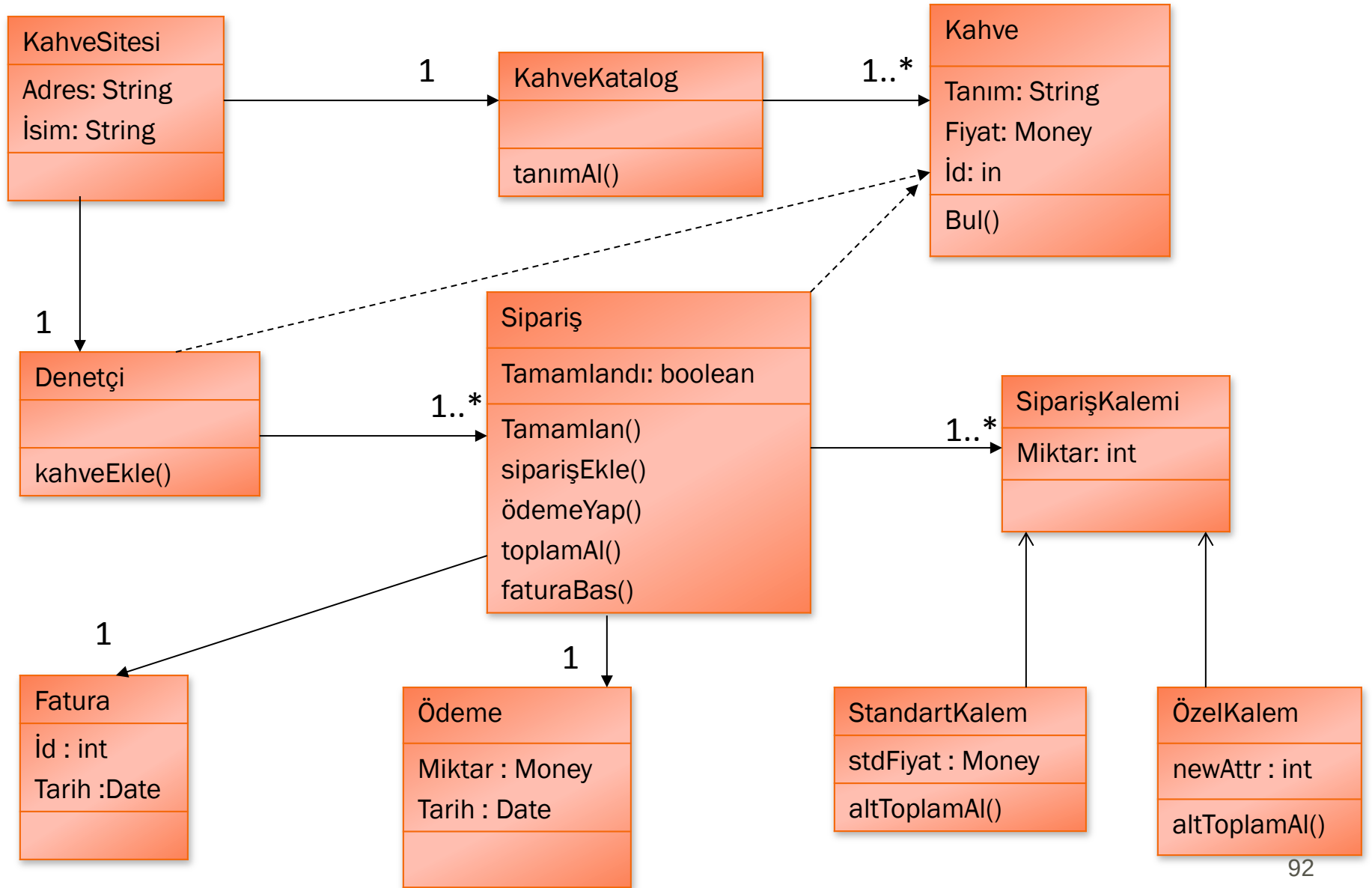
Görünürlük, nesnenin referansa sahip olması olarak tanımlanabilir. Dört çeşit görünürlük tipi bulunmaktadır. Bunlar;

- **Özellik görünürlük:** B nesnesi A nesnesinin bir özelliğidir. Bu durumda A,B ye sahiptir.
- **Parametre görünürlüğü:** B nesnesi A nesnesinin bir metodunda parametre olarak yer alır. Görünürlük sadece o metodun içinde ve çalıştığı sürece geçerlidir. Metot bitince görünürlük ortadan kalkar.
- **Yerel görünürlük:** B nesnesi A nesnesinin bir metodunda yerel değişken olarak yer alır. Görünürlüğün geçerliliği metodun çalışma süresi kadardır.
- **Global görünürlük:** B nesnesi ile A nesnesi aynı uzayda yer aldığı anda olan görünürlük tipidir. A ve B nesneleri var oldukça görünürlük devam eder.

Tasarım Sınıf Diyagramı

Senaryoları gerçekleştirmek üzere etkileşim diyagramlarının oluşturulmasına paralel olarak, yazılım sınıflarını gösteren tasarım sınıf diyagramı da çizilir. Tasarım sınıfı diyagramında yazılım sınıfları, sınıfların özellikleri, özelliklerin tipleri, metotları, metotların parametreleri ve erişim hakları gösterilir.

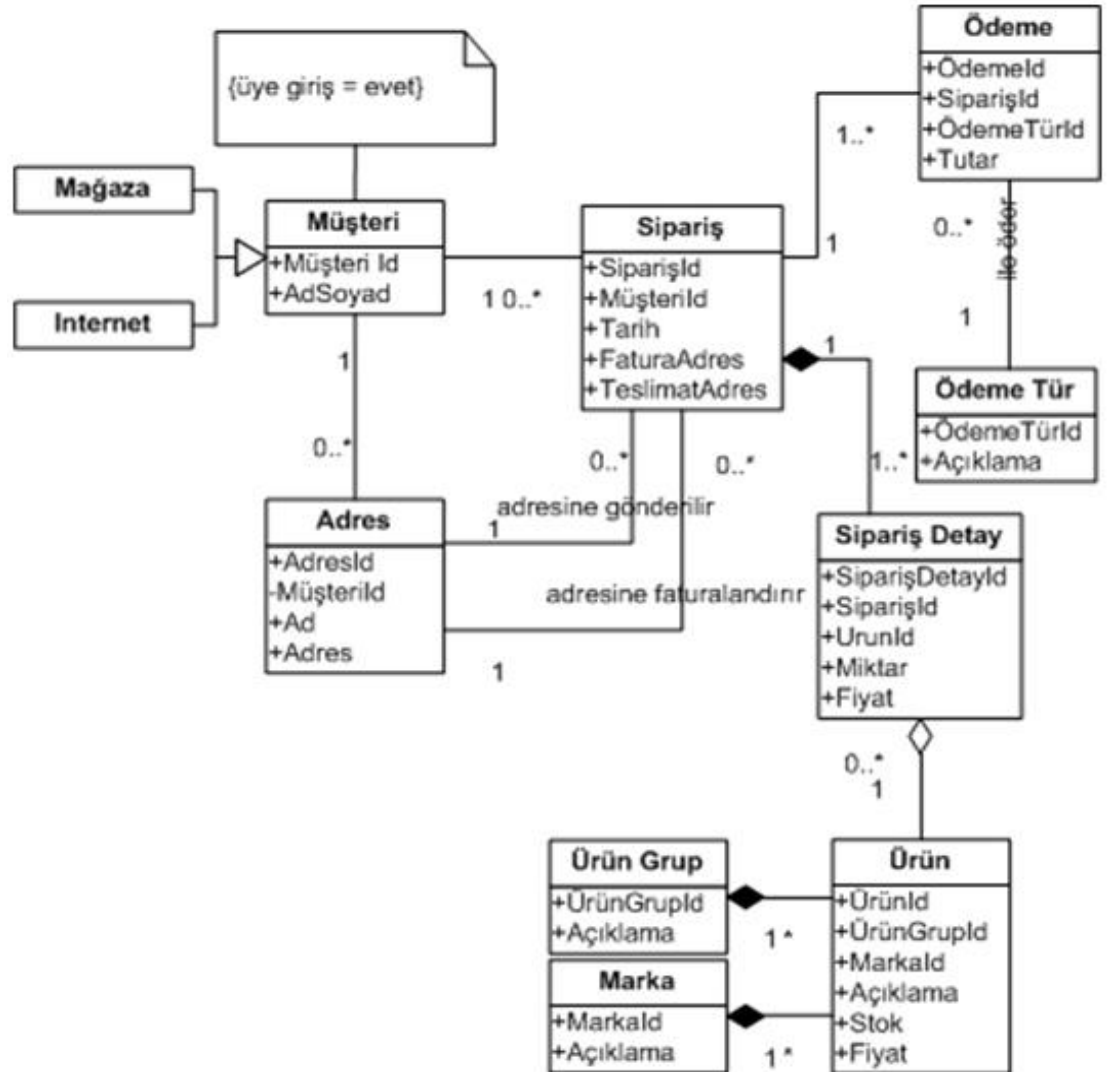
Yazılım sınıfları arasında bağımlılık, özellik görünürlüğü dışındaki görünürlük tipleri sonucunda oluşur. Sınıf diyagramında kesik çizgili ok ile gösterilir.



Örnek

Örnek: Bir e-ticaret uygulamasında sipariş

☞ Bir e-ticaret projesinin önemli nesnelerinden birisi olan sipariş ve onunla ilgili nesneler verilen UML diyagramında olduğu gibi modellenebilir.



Verilen diyagram aşağıdaki gibi yorumlanabilir:

- Bu projede iki tür müşteri bulunmaktadır; genelleştirme (generalization). Müşteriler ya doğrudan İnternet'ten kendileri sipariş verebilir veya bir mağazaya gidip oradan sipariş verebilirler.
- Müşteri nesnesi için bir koşul/kısıt (constraint) tanımlanmıştır. Bu koşula göre, ancak ve ancak siteye üye girişi yapmış kişiler sipariş verebilir.
- Müşteri ile sipariş arasında ilişki vardır. Bir müşterinin 0 veya daha fazla sayıda siparişi olabilir (0..*). Fakat bir sipariş ancak bir tek müşteriye (1) bağlı olabilir. Bir siparişte en az 1 ürün vardır (1..*) ve bir sipariş en az bir ödeme şekline sahiptir (1..*).
- Sipariş ile “sipariş detay” arasında oluşum (composition) bağıntısı vardır. Söz konusu sipariş silindiği zaman o siparişe ilişkin ayrıntı detay kayıtlar da silinir. “Sipariş detay” ile ürün arasında içerim (aggregation) bağıntısı vardır. Bu durumda sipariş detay kaydı silindiği zaman o kayıtlarla ilişkili ürün nesnesi silinmez.
- Bir müşterinin sistemde 0 (sıfır) veya daha fazla sayıda adresi tanımlı olabilir. Her siparişin bağlı olduğu iki adres vardır. Bu adreslerden biri fatura adresi diğeri teslimat adresi olarak kullanılır ve her sipariş için ancak 1 (bir) fatura adresi ve 1 (bir) teslimat adresi tanımlanabilir.